

Overlays of Peer to Peer Systems

Franck Petit

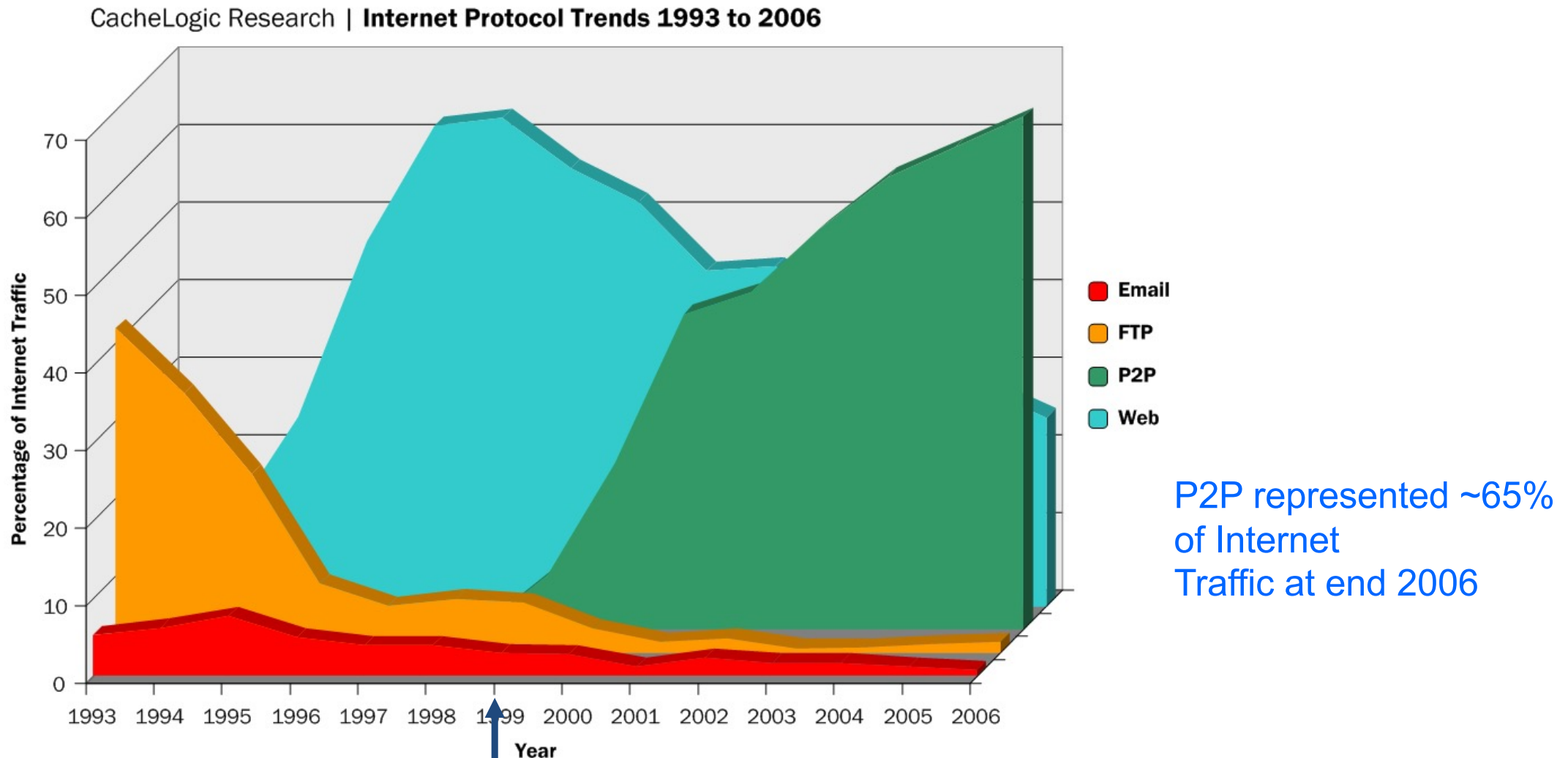
Lip6, UPMC

Mainly based on materials by:

- Yang Guo and Christoph Neumann, Corporate Research, Thomson Inc.
- Jon Crowcroft, Univ. of Cambridge
- Jim Kurose, Brian Levine, Don Towsley, UMASS
- Matthew Allen, Univ. of California Santa Barbara
- Manan Rawal, Bhaskar Gupta
- Alberto Montresor, UniTN
- Olivier Marin

- History, motivation and evolution
 - – History: Napster and beyond
 - What is Peer-to-peer?
 - Why Peer-to-peer?
- Brief P2P technologies overview
 - Unstructured P2P-overlays
 - Structured P2P-overlays

History, motivation and evolution



- 1999: *Napster*, first widely used P2P-application



1999: *Napster*, first widely used P2P application

The application:

- A P2P application for the distribution of mp3 files
 - Each user can contribute its own content

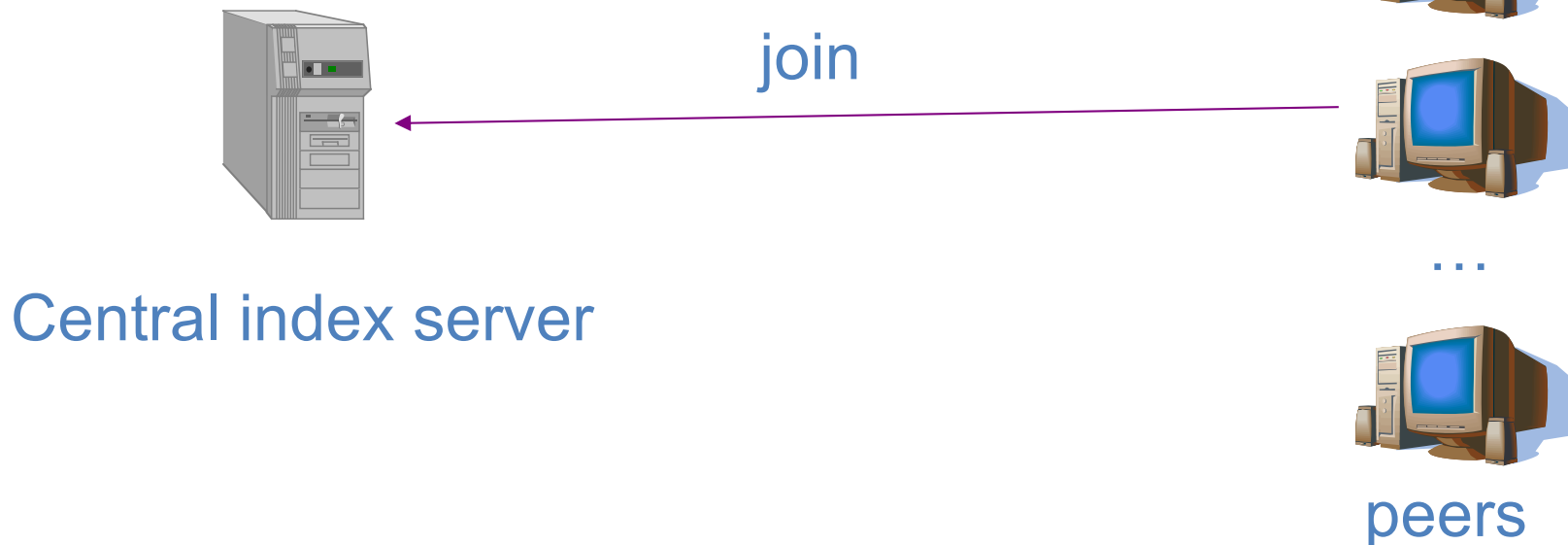
How it works:

- Central index server
 - Maintains list of all active peers and their available content
- Distributed storage and download
 - Client nodes also act as file servers
 - All downloaded content is shared

History, motivation and evolution - Napster (cont' d)

● Initial join

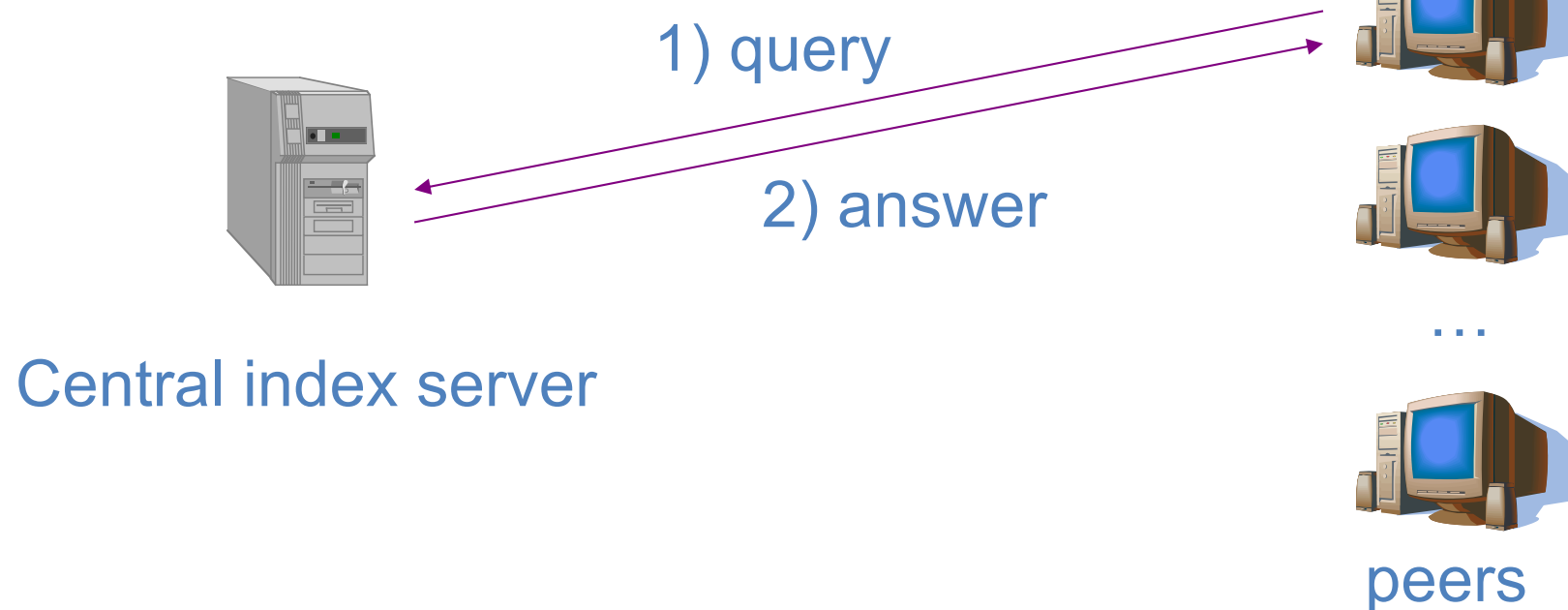
- Peers connect to Napster server
- Transmit current listing of shared files to server



History, motivation and evolution - Napster (cont' d)

- Content search

- Peers request to Napster server
- Napster server checks the database and returns list of matched peers

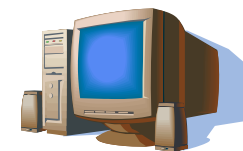
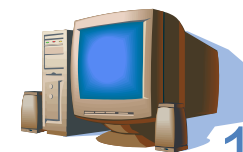
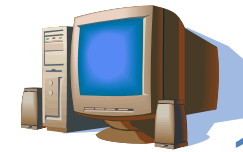
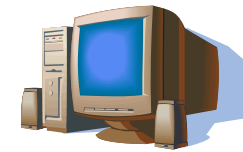
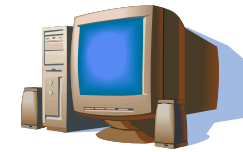
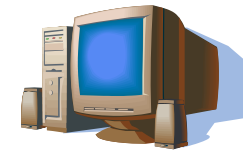


History, motivation and evolution - Napster (cont' d)

- File retrieval
 - The requesting peer contacts the peer having the file directly and downloads the it



Central index server



1) 2)

1) request

... 2) download

peers

History, motivation and evolution - File Download

- Napster was the first simple but successful P2P-application. Many others followed...

P2P File Download Protocols:

- 1999: Napster (closed in 2001)
- 2000: Gnutella, eDonkey
- 2001: Kazaa (closed in 2012)
- 2002: eMule, BitTorrent

- History, motivation and evolution
 - History: Napster and beyond
 - – What is Peer-to-peer?
 - Why Peer-to-peer?
- Brief P2P technologies overview
 - Unstructured P2P-overlays
 - Structured P2P-overlays

Definition of Peer-to-peer (or P2P)

- *DEFINITION 1* (early 2000s):
A peer-to-peer (or P2P) computer network is a network that relies primarily on the computing power and bandwidth of the participants in the network rather than concentrating it in a relatively small number of servers.
- *DEFINITION 2* (2009-):
A peer-to-peer (or P2P) is any **distributed network architecture** composed of **participants** that make a portion of their resources (such as processing power, disk storage or network bandwidth) directly available to other network participants, without the need for central coordination instances.

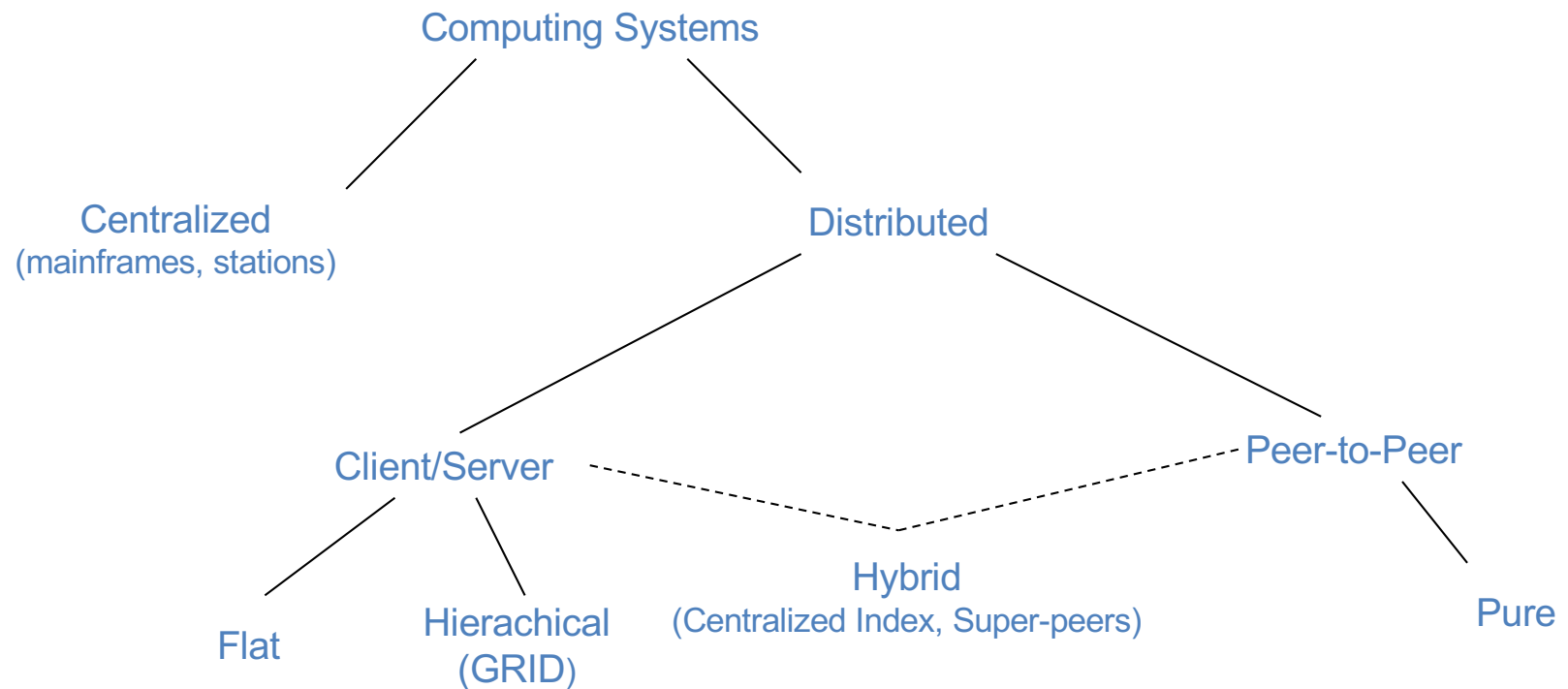
Taken from the wikipedia free encyclopedia - www.wikipedia.org

Definition of Peer-to-peer (or P2P)

- *DEFINITION 3* (2016-):
Peer-to-peer (P2P) computing or networking is a distributed application architecture that **partitions tasks or workloads between peers**. Peers are equally privileged, equipotent participants in the application. They are said to form a peer-to-peer network of nodes.

Taken from the wikipedia free encyclopedia - www.wikipedia.org

The Big Picture

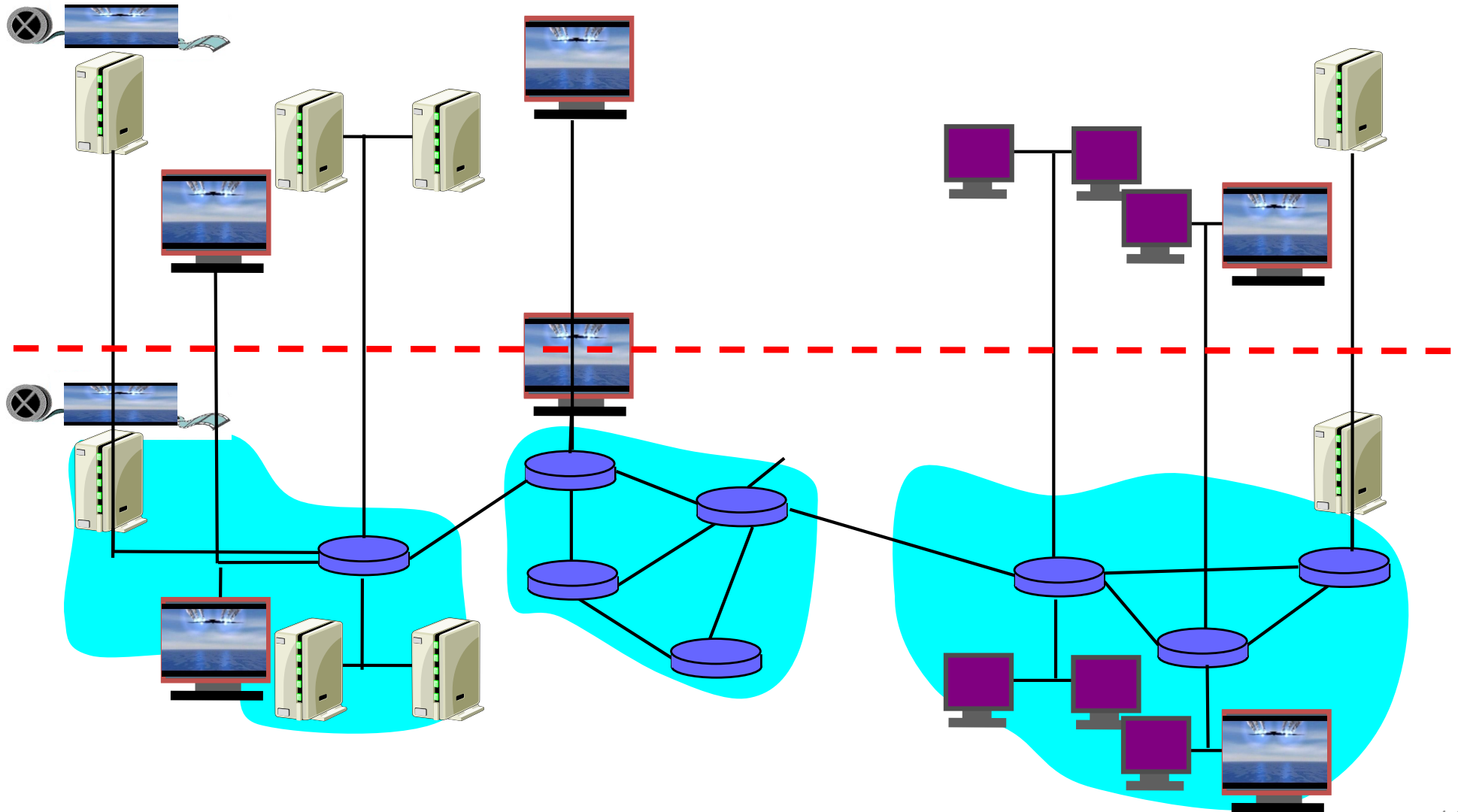


Peer to peer systems

- Assumed to be with **no distinguished role**
- So no single point of bottleneck or failure
- However, this means they need **distributed algorithms** for
 - Connection protocol
 - Service discovery (name, address, route, metric, etc)
 - Neighbour status tracking
 - Application layer routing (based possibly on content, interest, etc)
 - Resilience, handling link and node failures
 - etc

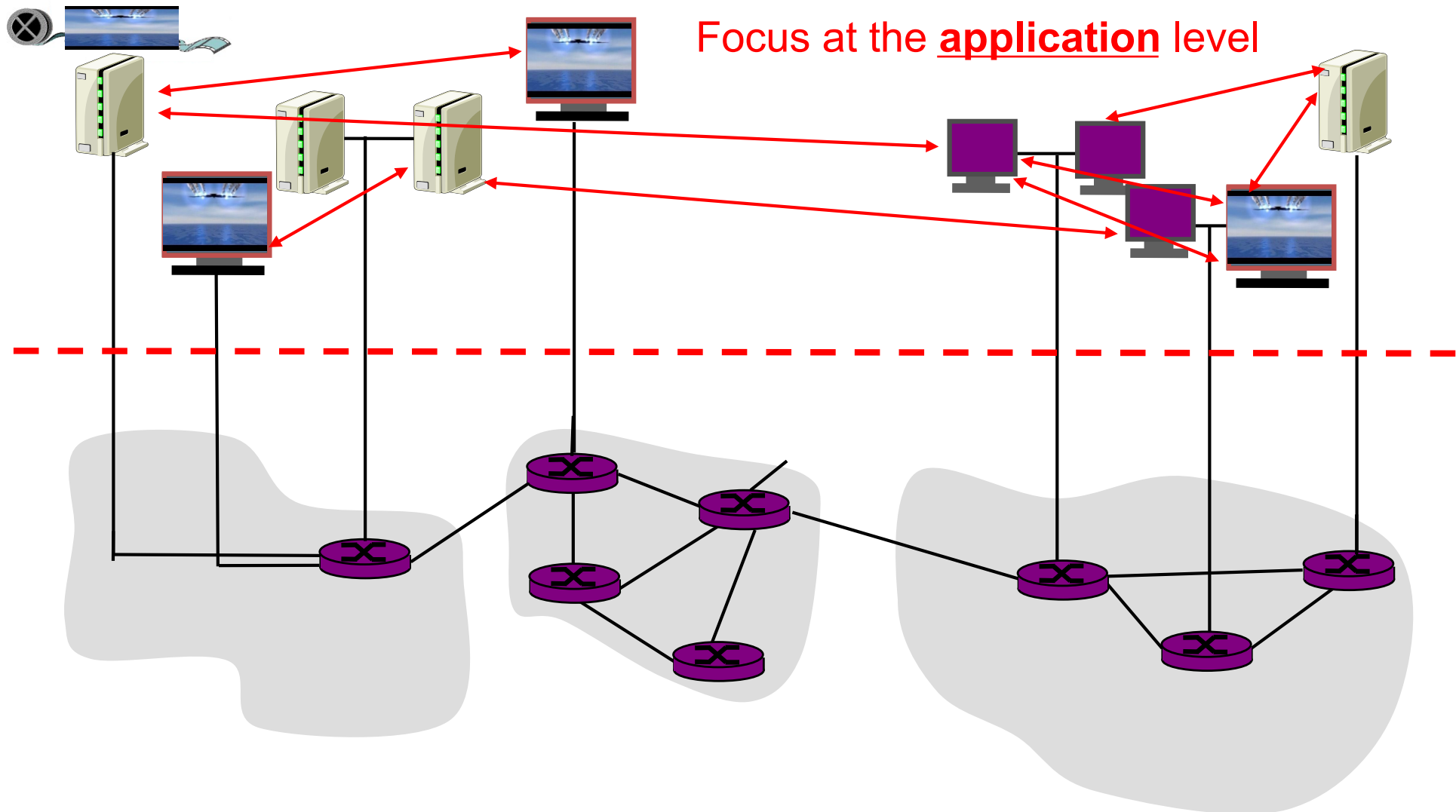
Overlays and peer 2 peer systems

- P2P systems rely on **overlays** structure

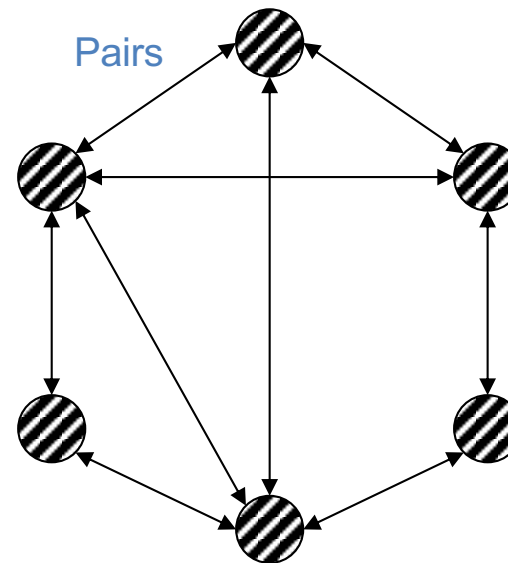
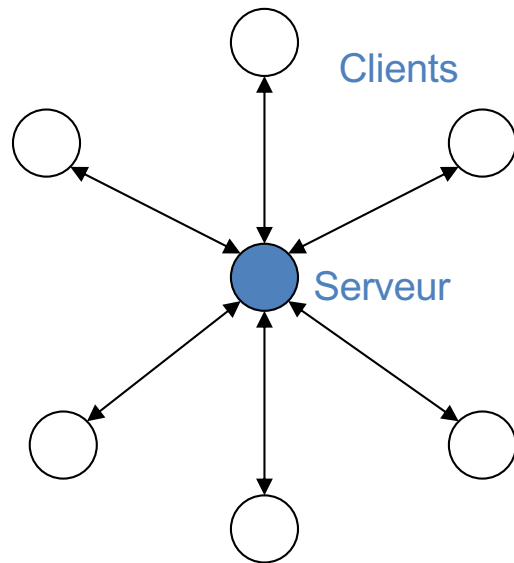


Overlays and peer 2 peer systems

- P2P systems rely on **overlays** structure



C/S vs. P2P



C/S vs. P2P

- Gérés
 - Configurés
 - Recherche de services
 - Hiérarchique
 - Ressources statiques
 - Cycle de vie lié au serveur
 - Centré IP
 - Nommage basé sur le DNS
 - Communications type RPC/RMI
 - Synchrones
 - Asymétrique
 - Axé sur des modèles de liaison et d'intégration du langage de programmation (stub IDL/XDR, compilateurs, etc...)
 - Sécurité de type Kerberos : acl, crypto
- Auto-Gérés
 - Ad-hoc
 - Découverte de services
 - Maillage
 - Ressources volatiles
 - Cycle de vie autonome
 - Non restrictif à IP
 - Nommage spécifique
 - Communication par messages
 - Asynchrone
 - Symétrique
 - Axé sur la localisation de services, localisation du contenu, routage applicatif
 - Anonymat, haute disponibilité
 - Plus difficile à maîtriser

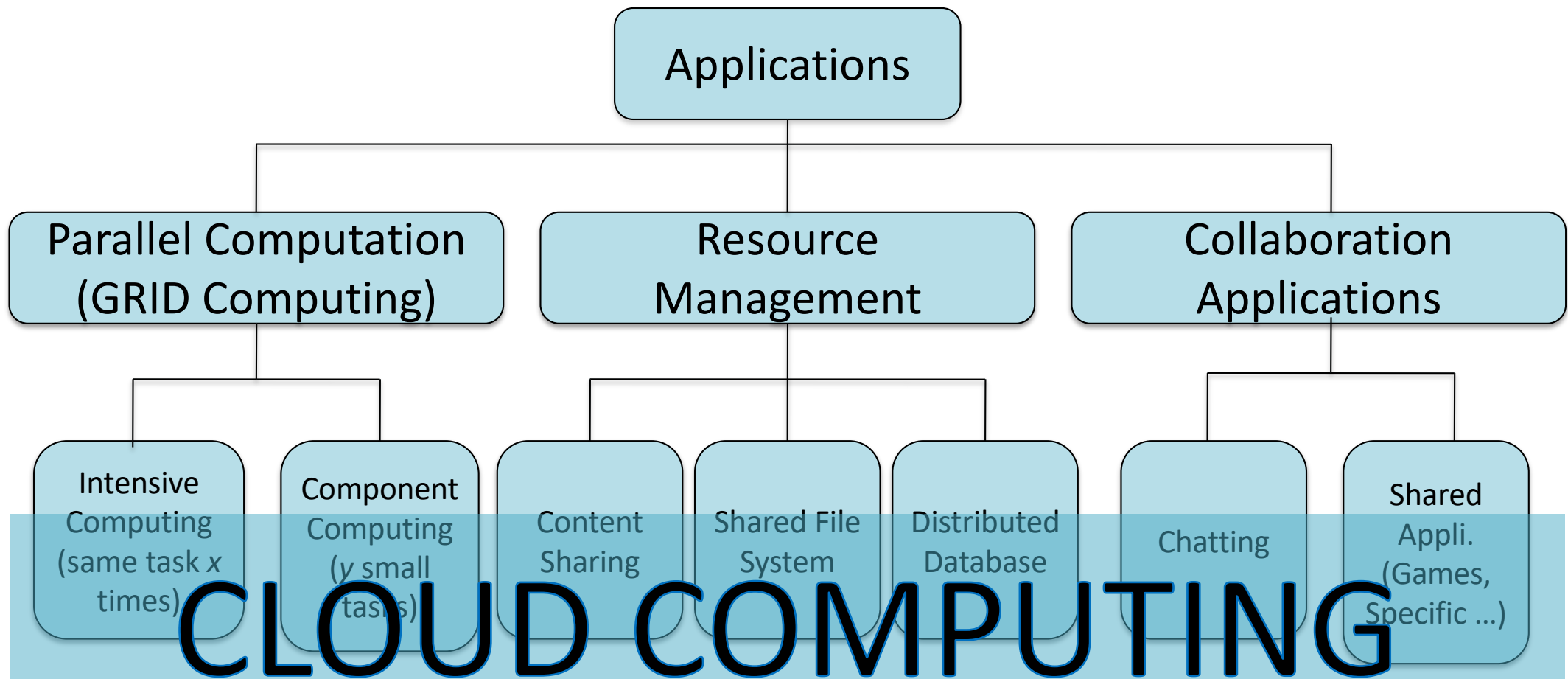
P2P : Nouveaux besoins algorithmiques

- Découverte de services (nom, adresse, route, métrique, ...)
- Recherche de voisins
- Routages spécifiques, de niveau applicatif
- Rémanence, récupération sur faute de liaison ou d'exécution
- Maîtrise de la confidentialité

P2P : Contraintes algorithmiques

- Structure du système
 - totalement décentralisée
 - dynamique
- Diffusion à redéfinir
 - Pas d'ensemble identifié de processus : nombre inconnu et variable
 - Topologie d'interconnexion peut aussi être inconnue et dynamique
 - Symétrie entre sites (client **et** serveur)
- Modèle de panne **avec** fautes

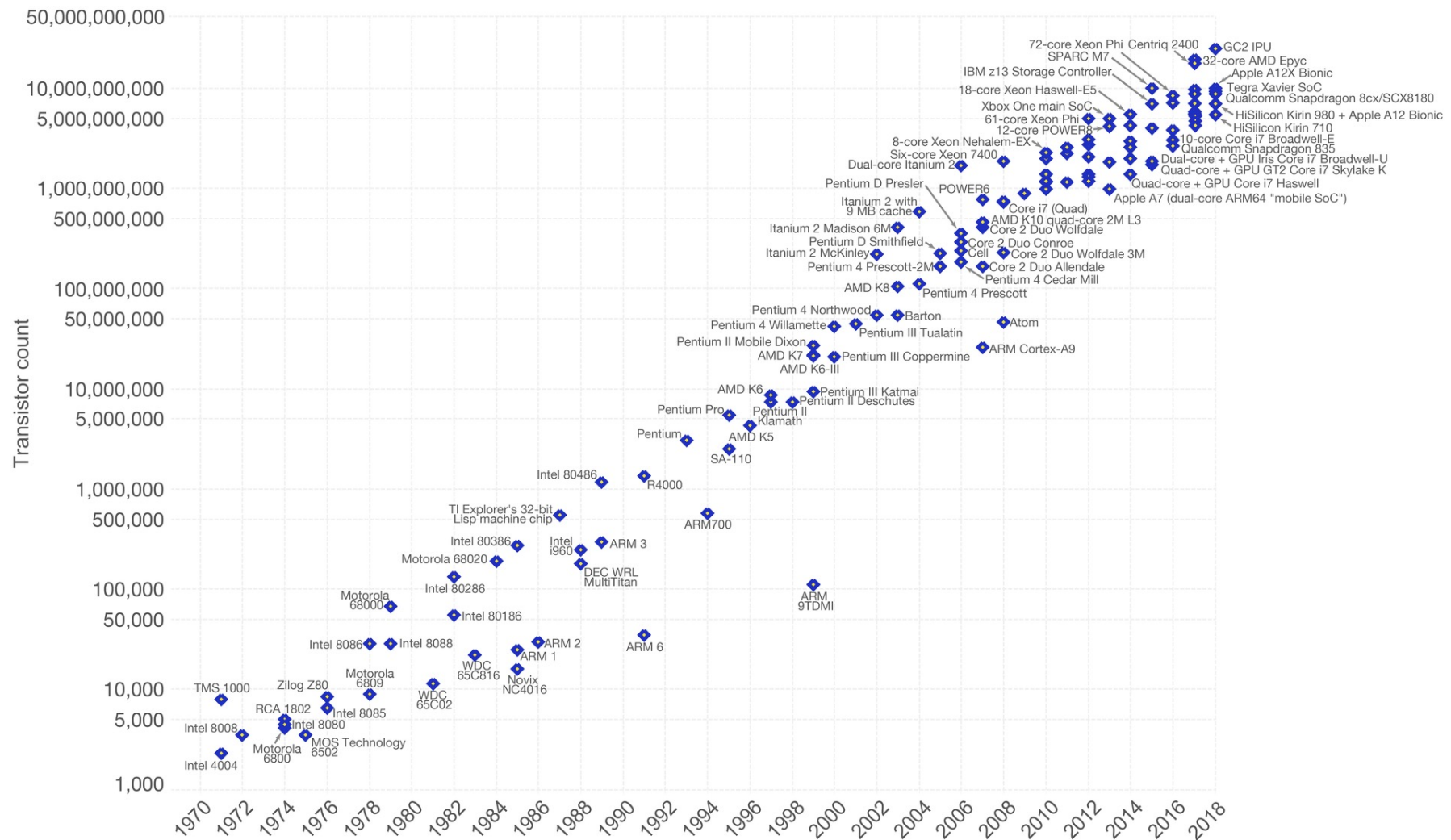
Taxonomy of Applications



Computing Power

Moore's Law – The number of transistors on integrated circuit chips (1971-2018)

Moore's law describes the empirical regularity that the number of transistors on integrated circuits doubles approximately every two years. This advancement is important as other aspects of technological progress – such as processing speed or the price of electronic products – are linked to Moore's law.



Data source: Wikipedia (https://en.wikipedia.org/wiki/Transistor_count)

The data visualization is available at [OurWorldinData.org](https://ourworldindata.org). There you find more visualizations and research on this topic.

Licensed under [CC-BY-SA](#) by the author Max Roser.

Taxonomy of Applications

- P2P-File download
 - Napster, Gnutella, KaZaa, eDonkey, Bittorrent...
- P2P-Communication
 - VoIP, Skype, Messaging, ...
- P2P-Video-on-Demand
- P2P-Computation
 - GRID, scientific computation (seti@home, XtreemOS, BOINC,...)
- P2P-Chat
 - Telegram (2013)
 - ShazzleChat (2013)
 - Signal Messenger (2018)
- P2P-Streaming
 - PPLive, End System Multicast (*ESM*), *iTunes* (2003), Deezer (2007),...
- P2P-Gaming
 - WOW, City of Heroes,...
- P2P-Blockchain
 - BTC (BitCoin) (2009)
- P2P-Visio conference
 - Jitsi Meet (2021)
 - Jami (GNU)
 - talky, Team Video, TOX, Retroshare,...

History, motivation and evolution - Applications

- P2P is not restricted to file download!

P2P Protocols:

- 1999: Napster, End System Multicast (ESM)
- 2000: Gnutella (Bearshare, LimeWire), eDonkey
- 2001: FastTrack (Kazaa)
- 2002: eMule, BitTorrent (Vuze)
- 2003: Skype
- 2004: PPLive
- 2006: TVKoo, TVAnts, PPStream, SopCast, Video-on-Demand, Gaming
- 2010---: messagerie instantanée, visio conference

Application type:

File Download

Streaming

Telephony

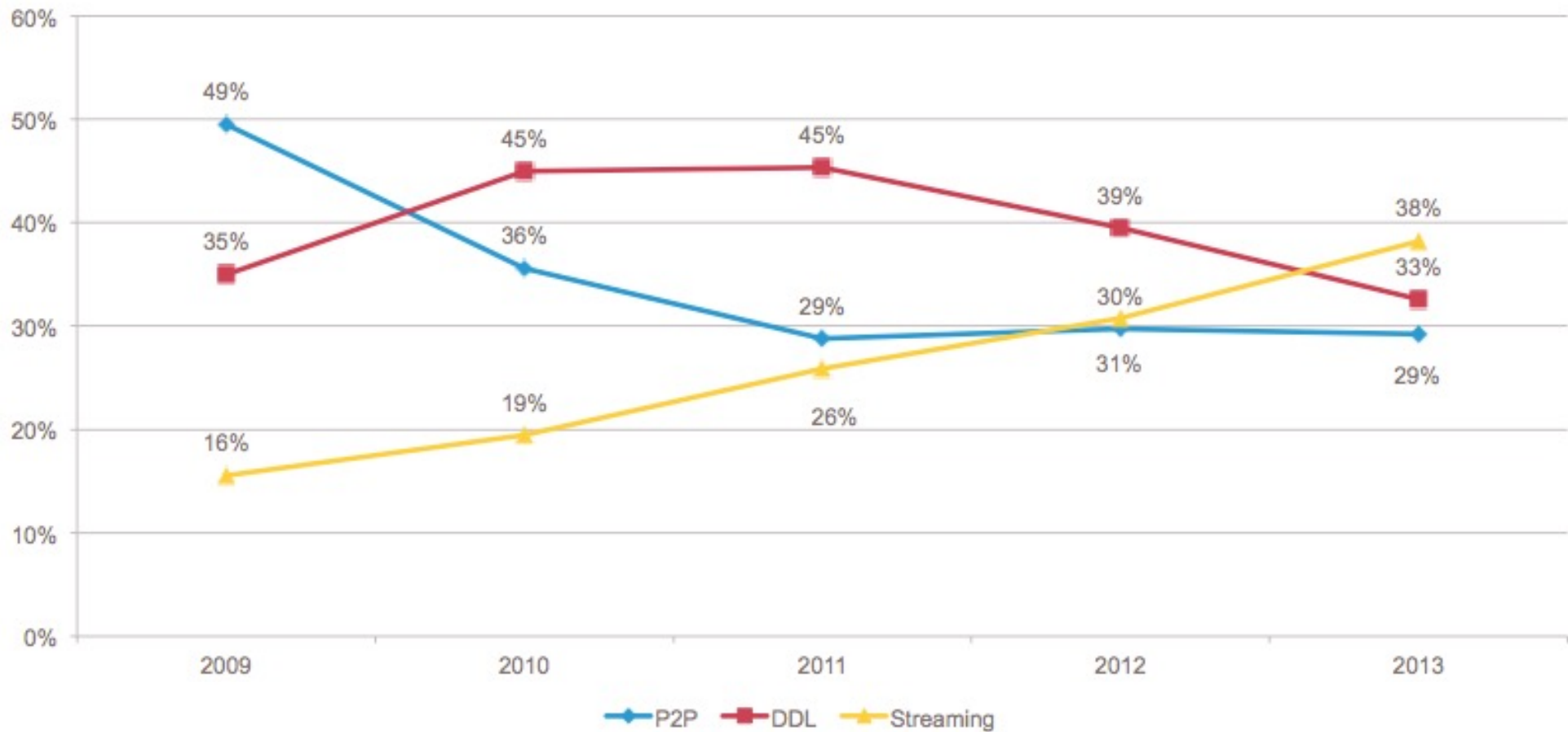
Video-on-Demand

Gaming

Visio conference

Evolution

Évolution de la répartition des usages (nombre de pages vues) par protocole depuis 2009

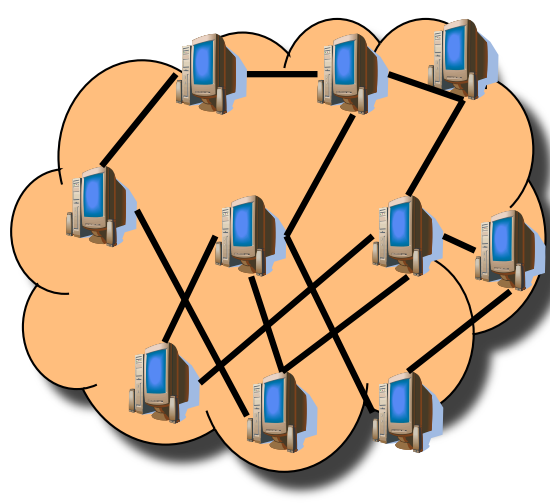


- History, motivation and evolution
 - History: Napster and beyond
 - What is Peer-to-peer?
 - – Why Peer-to-peer?
- Brief P2P technologies overview
 - Unstructured P2P-overlays
 - Structured P2P-overlays

Why is P2P so successful?

- Scalable – It's all about sharing resources
 - No need to provision servers or bandwidth
 - Each user brings its own resource
 - *e.g.* resistant to flash crowds
 - flash crowd = a crowd of users all arriving at the same time

capacity



Resources could

- Files to share;
- Upload bandwidth;
- Disk storage;...

Why is P2P so successful? (cont' d)

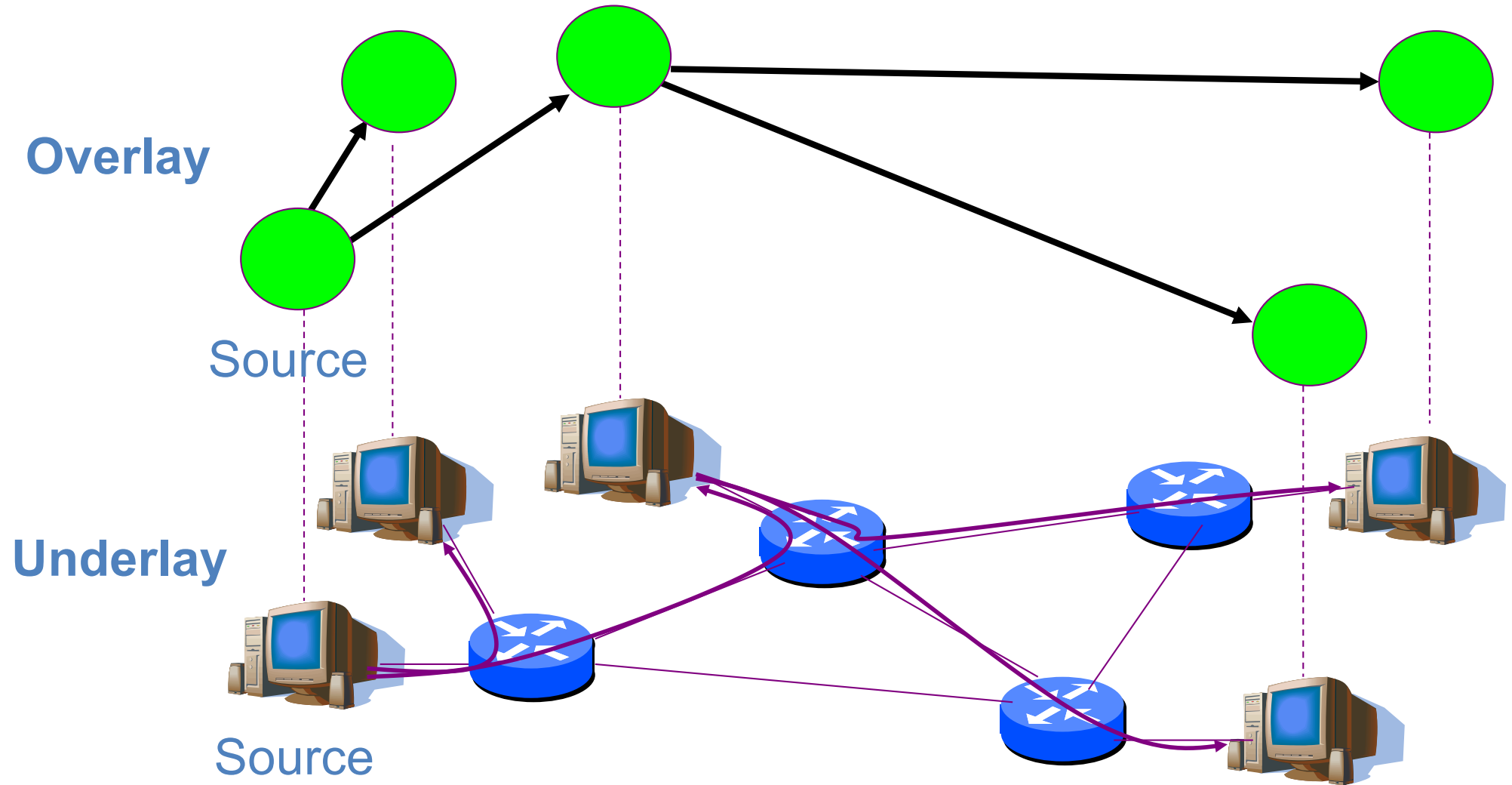
- Cheap - No infrastructure needed
- Everybody can bring its own content (at no cost)
 - Homemade content
 - Ethnic content
 - Illegal content
 - But also *legal* content
 - ...
- High availability – Content accessible most of the time (replication)

- History, motivation and evolution
 - History: Napster and beyond
 - What is Peer-to-peer?
 - Why Peer-to-peer?
- • Brief P2P technologies overview
 - Unstructured P2P-overlays
 - Structured P2P-overlays

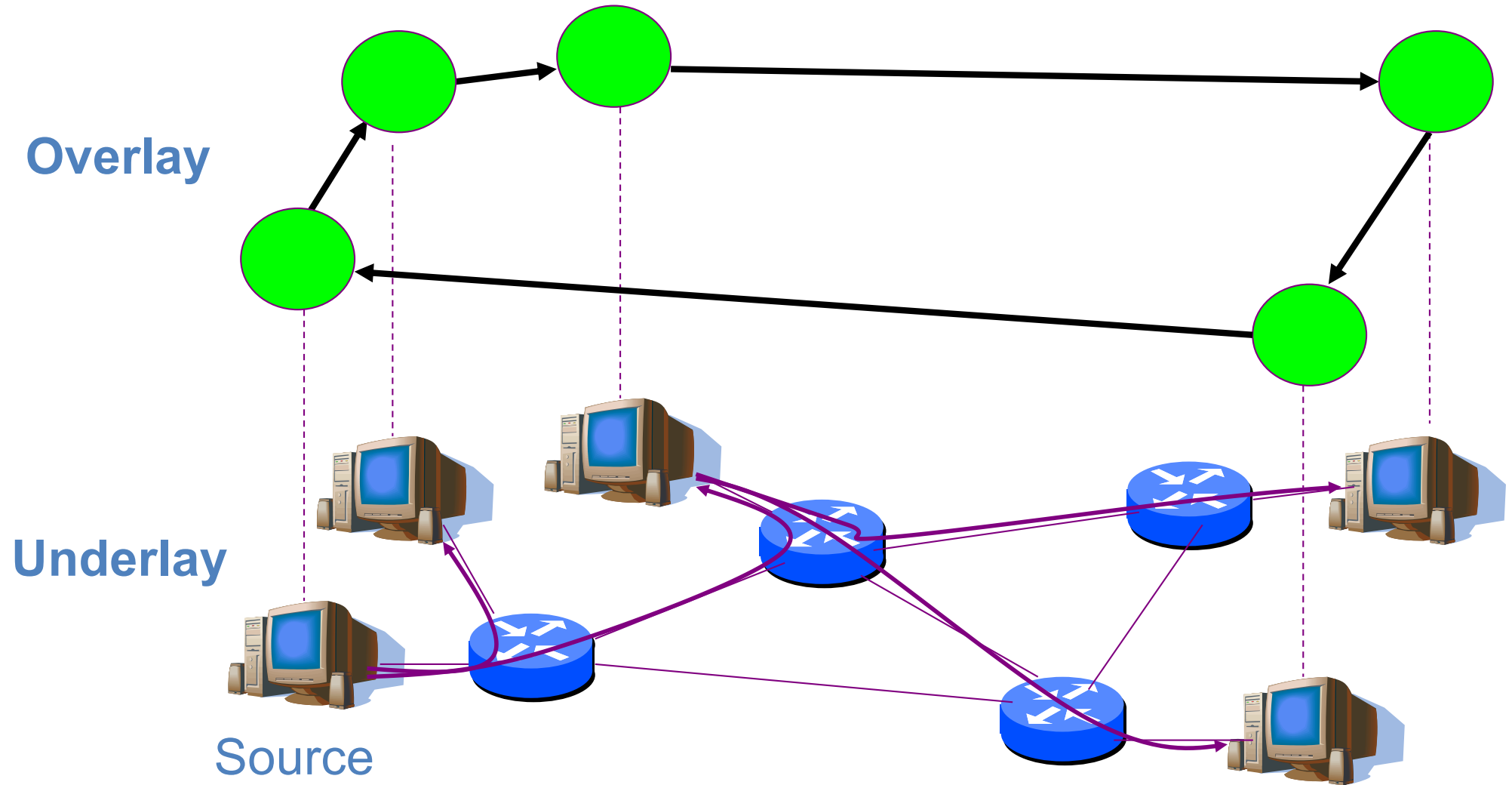
P2P-Overlay

- Build graph at application layer, and forward packet at the application layer
- It is a *virtual* graph
 - Underlying physical graph is transparent to the user
 - Edges are TCP connection or simply a entry of an neighboring node's IP address
- The graph has to be continuously maintained (e.g. check if nodes are still alive)

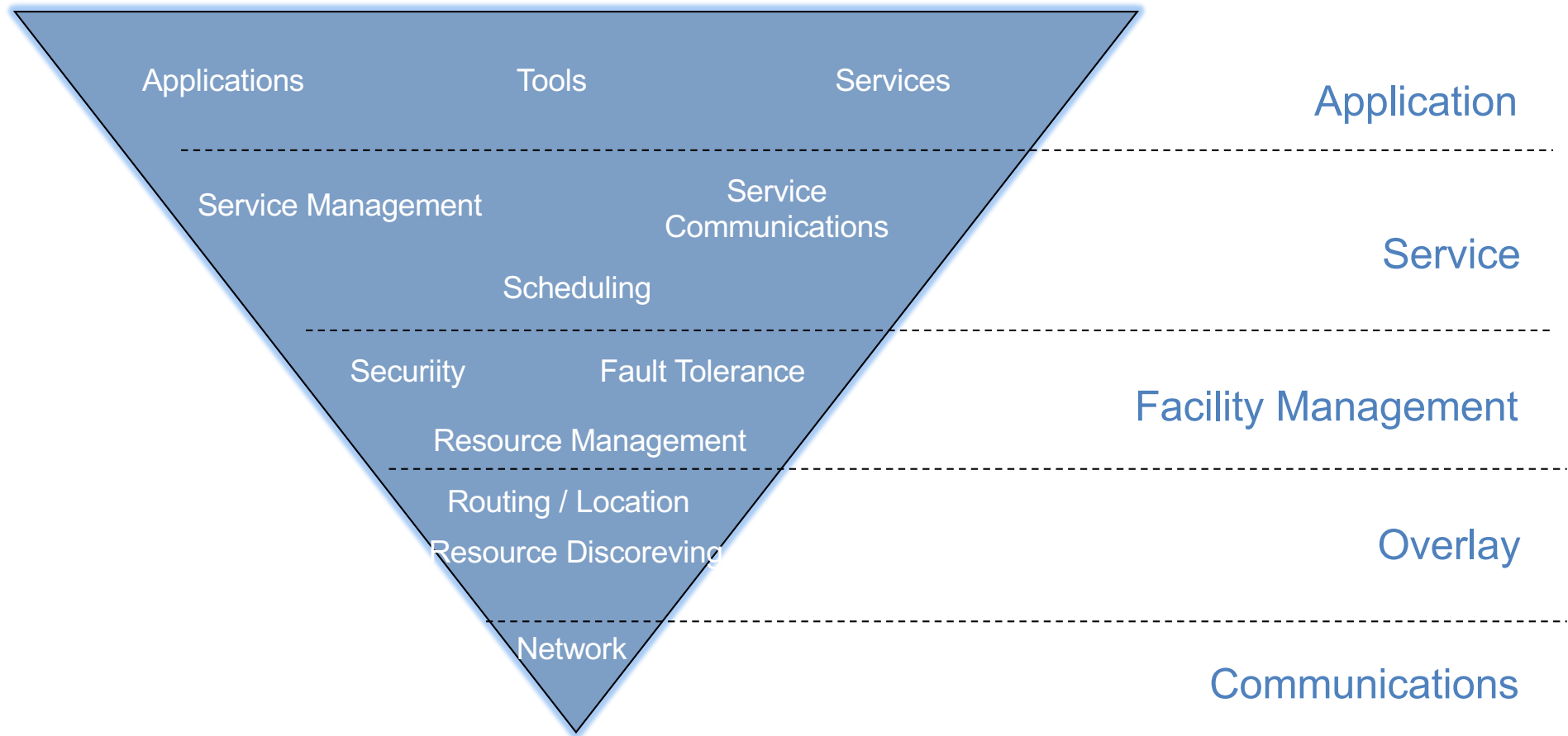
P2P-Overlay (cont' d)



P2P-Overlay (cont' d)



P2P-Overlay Abstraction



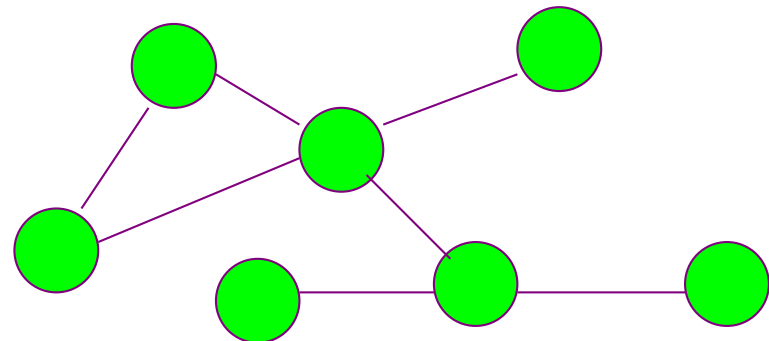
The P2P enabling technologies

- Unstructured P2P-overlays
 - Generally random overlay
 - Used for content download, telephony, streaming
- Structured P2P-overlays
 - Distributed Hash Tables (DHTs)
 - Used for node localization, content download, streaming

- History, motivation and evolution
 - History: Napster and beyond
 - What is Peer-to-peer?
 - Why Peer-to-peer?
- Brief P2P technologies overview
 - – Unstructured P2P-overlays
 - Structured P2P-overlays

Unstructured P2P-overlays

- Unstructured P2P-overlays do not really care how the overlay is constructed
 - Peers are organized in a random graph topology
 - *e.g.*, new node randomly chooses three existing nodes as neighbors
 - Flat or hierarchical
 - Build your P2P-service based on this graph
- Several proposals
 - Gnutella
 - KaZaA/FastTrack
 - BitTorrent



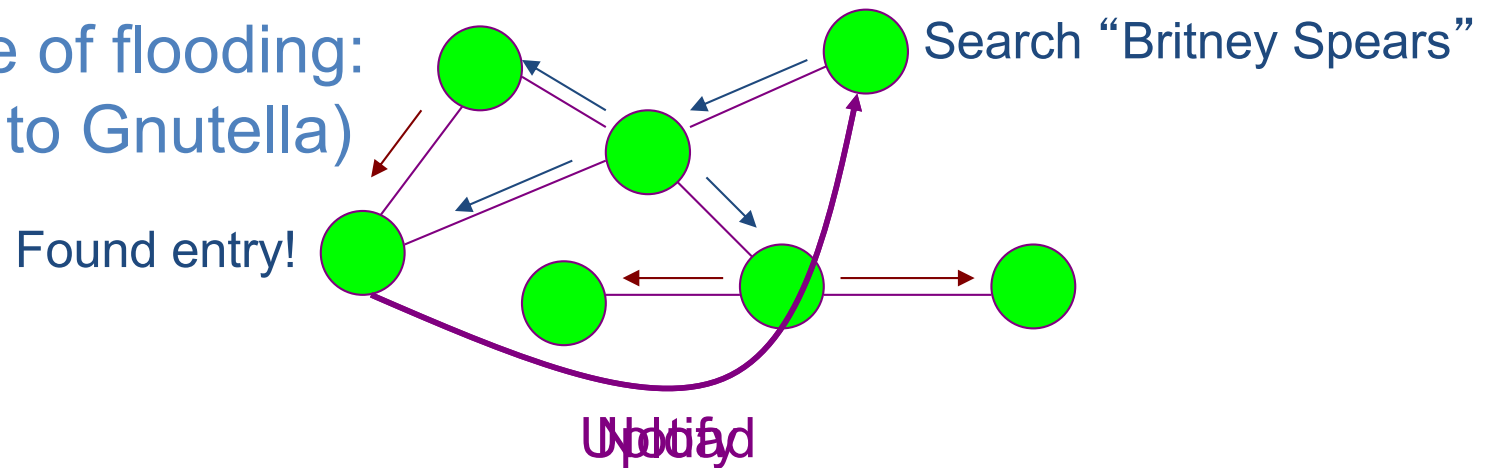
Unstructured P2P-overlays (cont' d)

- Unstructured P2P-overlays are just a framework, you can build many applications on top of it
- “Pure” P2P
- Unstructured P2P-overlays pros & cons
 - Pros
 - Very flexible: copes with dynamicity
 - Supports complex queries (conversely to structured overlays)
 - Cons
 - Content search is difficult: There is a tradeoff between generated traffic (overhead) and the horizon of the partial view

One Example of usage of unstructured overlays

- Typical problem in unstructured overlays: How to do content search and query?
 - Flooding

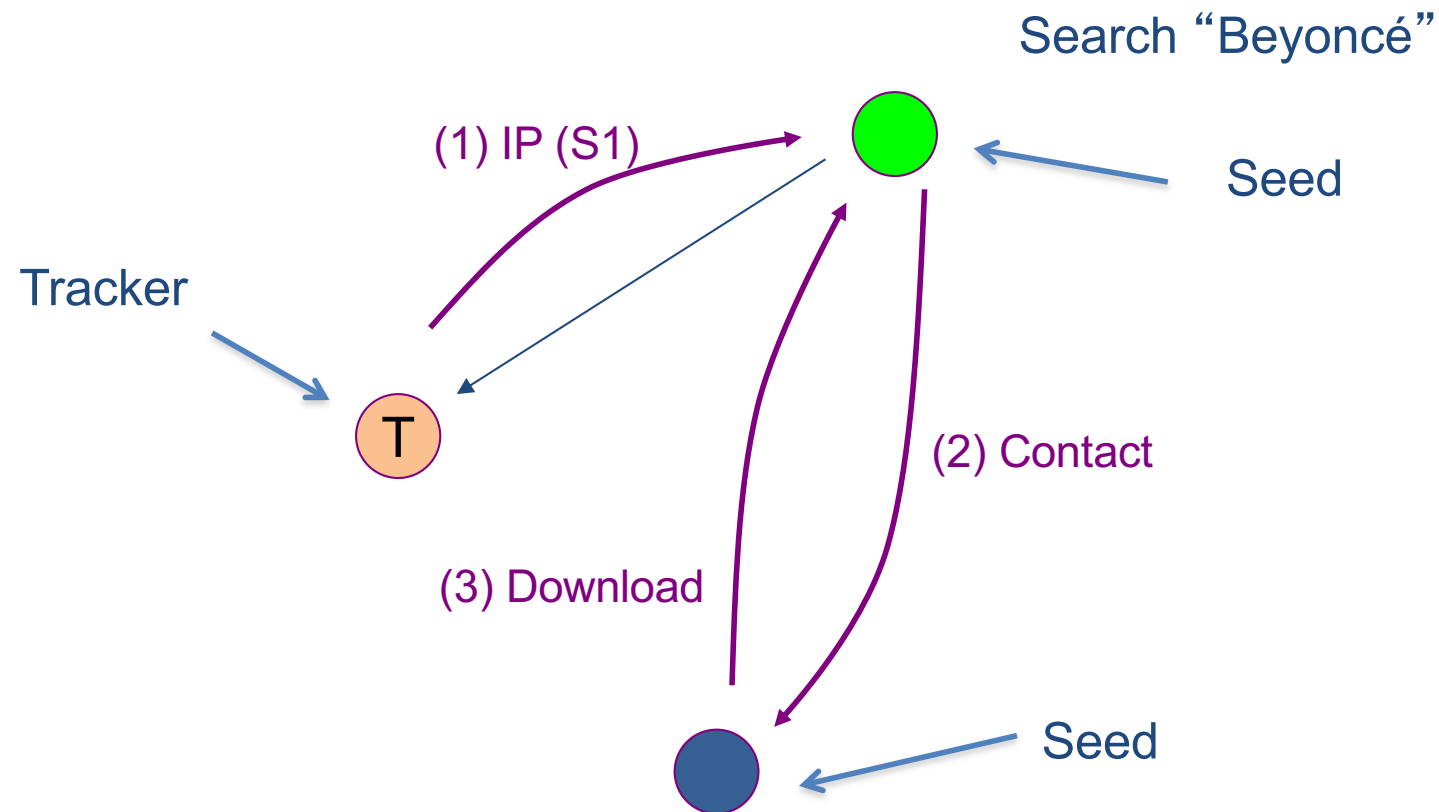
Example of flooding:
(similar to Gnutella)



- Limited Scope, send only to a subset of your neighbors
- **Time-To-Live**, limit the number of hops per messages

Another Example of usage of unstructured overlays

- Bittorrent



Semi-structuration : Freenet

Stockage persistant de données et services [Freenet00]

Nœuds/Données identifiés par une clé binaire (fonction hash)

Identifiant de nœud : $\text{NodeID}(\text{utilisateur}) = \text{hash}(@\text{IP})$

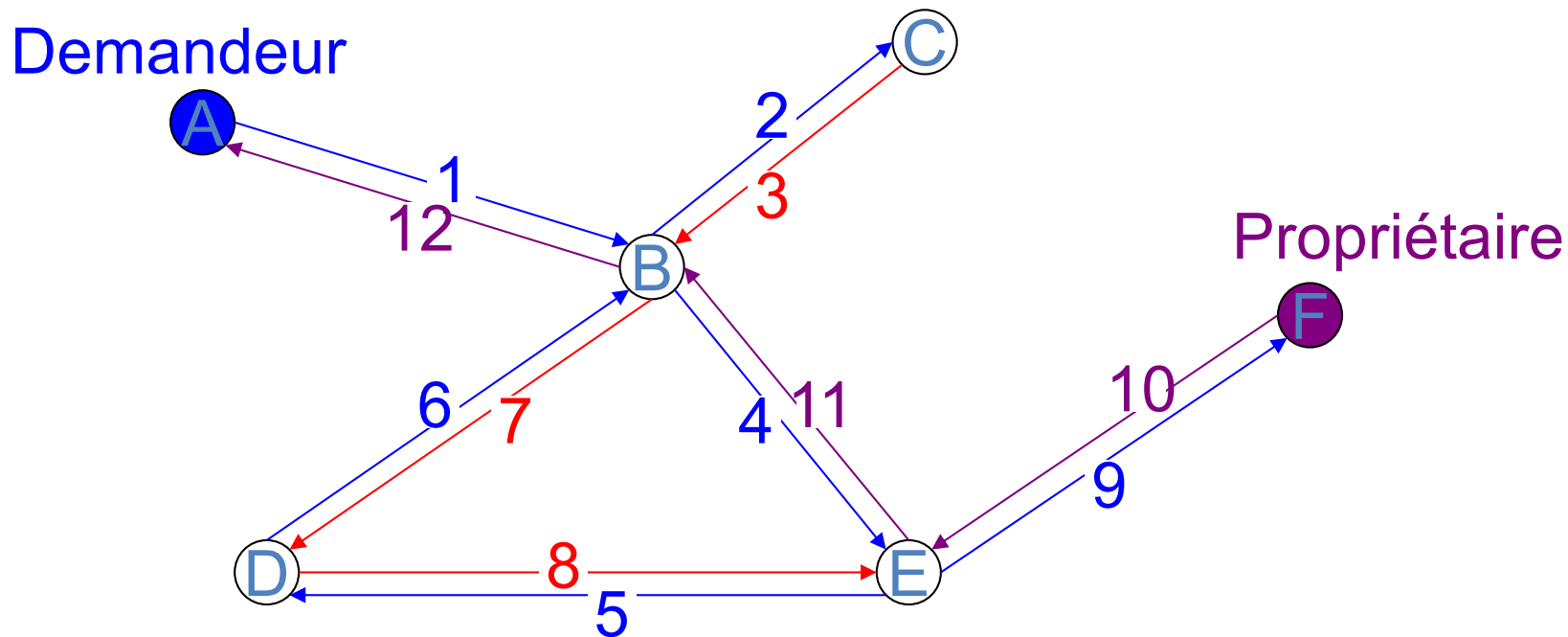
Identifiant (*clé*) de fichier : $\text{FileID}(\text{fichier}) = \text{hash}(\text{contenu})$

Gestion de tables de routage

Construction de voisinages

⇒ Connaissance approximative du contenu des nœuds proches

Semi-structuration : Freenet



Séquence typique de routage de requête

Dissémination de proche en proche

Gestion de cul-de-sac (3) et de boucle (7) (Depth-First Token Circulation)
jusqu'à une certaine distance (idem TTL)

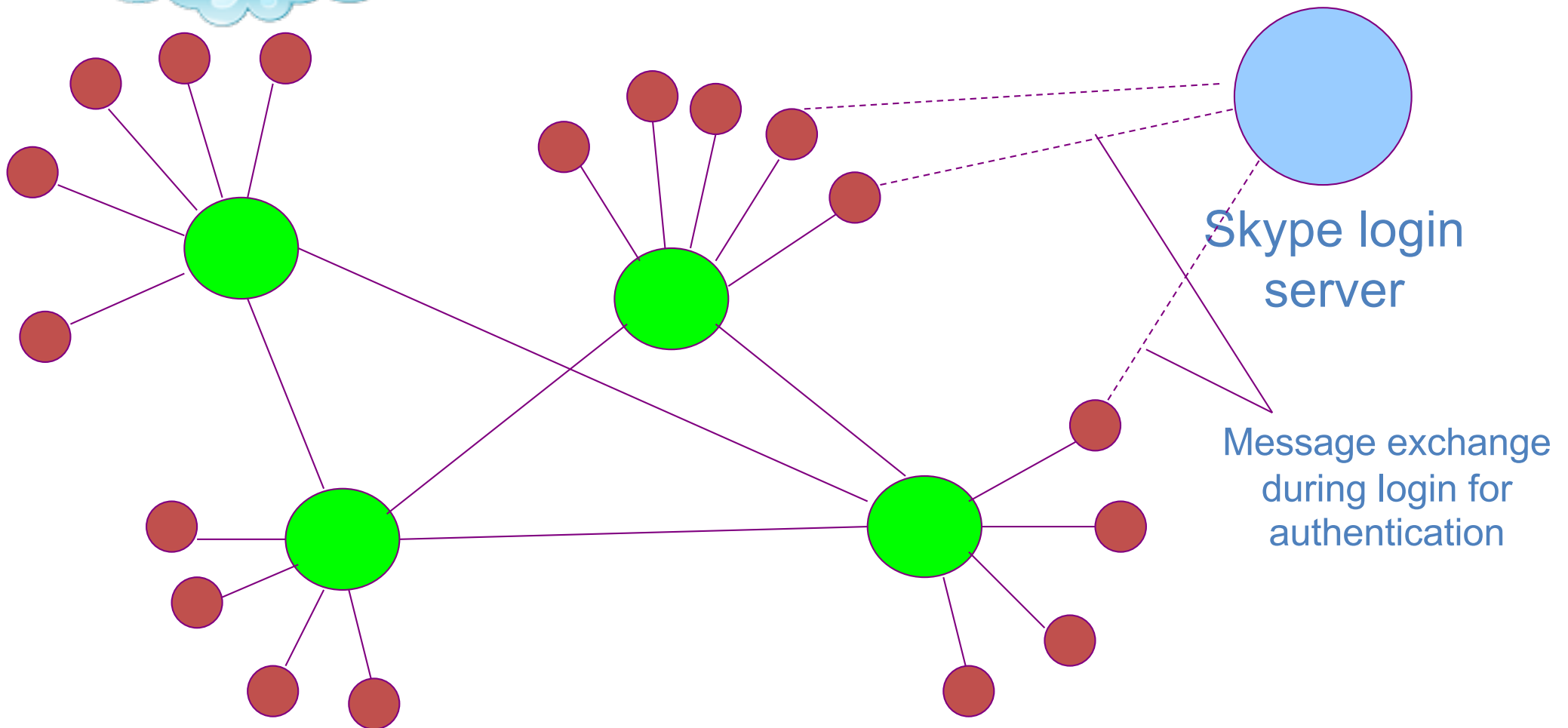


Collaboration

- Transfert de communications (VoIP)
- Pairs partagent leur bande passante
- Annuaire totalement décentralisé
- Routage au moyen de super-nœuds
- 246 millions d'utilisateurs (2014)
- Gros problèmes de sécurité



- Super Nodes (SN)
 - Connect to each other, building a flat unstructured overlay (similar to the Gnutella overlay)
- Ordinary Nodes (ON)
 - Connect to Super Nodes that act as a directory server (similar to the index server in Napster)
- Skype login server
 - Only central component
 - Stores and verifies usernames and passwords
 - Stores the buddy list





- Each node keeps a host cache with a list of Super Nodes IP-addresses
 - Up to 200 entries
- Some Super Nodes IP-addresses are hard-coded
 - Super Nodes provided by Skype
- These lists are used to locate a nodes Super Node at login



How is the overlay constructed? - Super Node Lists

- Super Nodes are index servers
 - I.e. index of locally connected Skype users (and their IP addresses)
- If buddy is not found in local index of a Super Node
 - Spread node search to neighboring Super Nodes ... by flooding (not clear the details, probably similar to Gnutella)



How is the overlay constructed? - Super Node Lists

- Contact login server and authenticate
- Advertise your presence to other peers
 - Contact a Super Node
 - Contact your buddies (through Super Node), and notify your presence

- History, motivation and evolution
 - History: Napster and beyond
 - What is Peer-to-peer?
 - Why Peer-to-peer?
- Brief P2P technologies overview
 - Unstructured P2P-overlays
 - – Structured P2P-overlays

Structured P2P-overlays

- Motivation
 - Locate content efficiently
- Solution – DHT (Distributed Hash Table)
 - Particular nodes hold particular keys
 - Locate a key: *route search request to a particular node that holds the key*
 - Representative solutions
 - *CAN, Pastry/Tapestry, Chord, etc.*

Challenges to Structured P2P-overlays

- Load balance
 - spreading keys evenly over the nodes.
- Decentralization
 - no node is more important than any other.
- Scalability
 - Lookup must be efficient even with large systems
- Peer dynamics
 - Nodes may come and go, may fail
 - Make sure that “the” node responsible for a key can always be found

Key Issue: Distributed Indexing

Tree-Based

- Pros:
 - Easy and well-known techniques
 - $O(\log n)$ operations
 - Equality & **Inequality Queries**
search for $d = \text{val}$, search for $d < \text{val}$, search for ' D^* ',...
- Cons:
 - Node hierarchy / Specialization of nodes (root, sub*-root, leaves, etc.)
 - Bottleneck
 - Fault-prone, lack of dynamic...

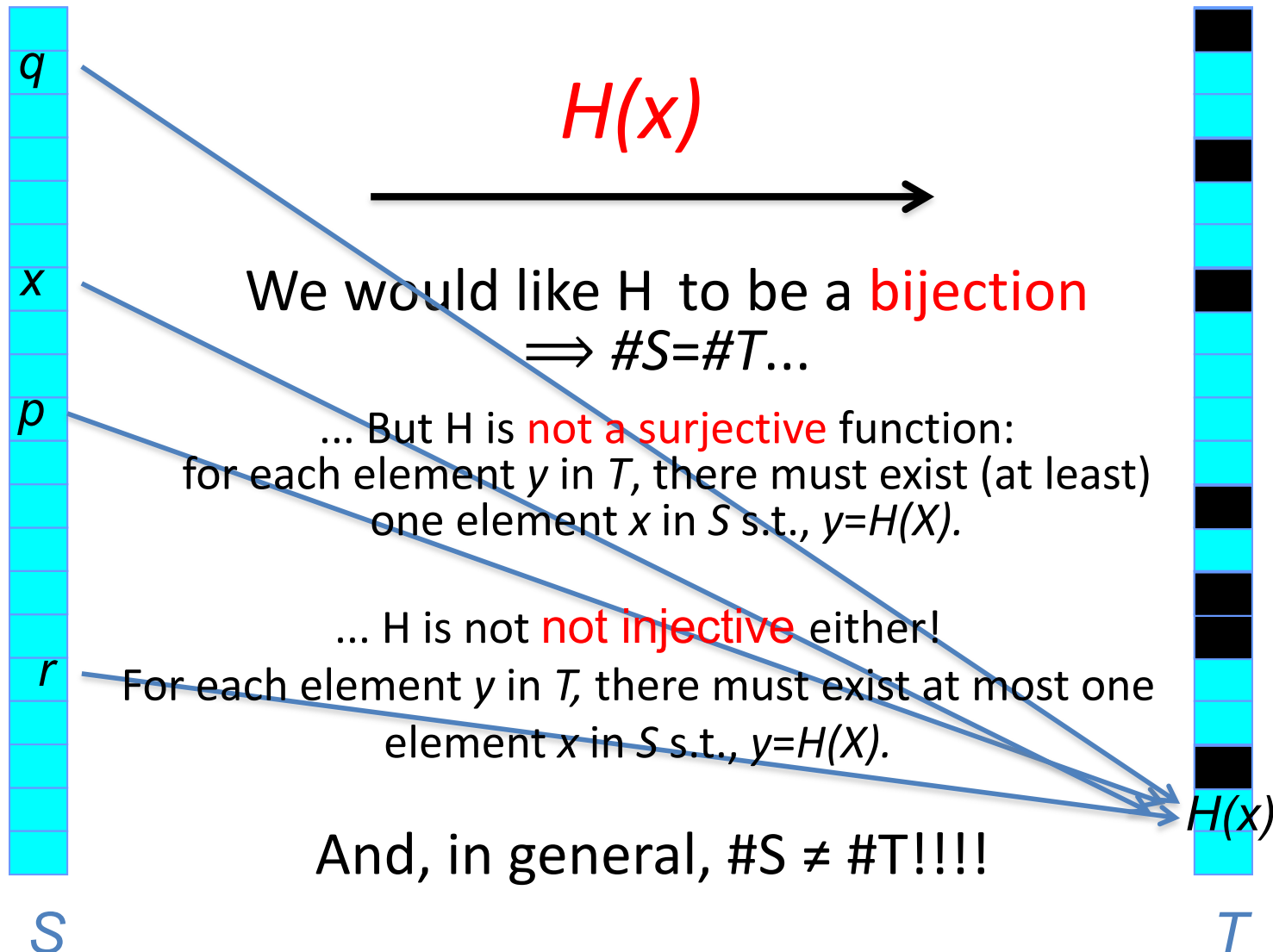
Key Issue: Distributed Indexing

Hash Function

- Pros:
 - Easy and well-known techniques
 - (Theoretically) $O(1)$ operation
 - Fault-tolerance and dynamic-maintenance
- Cons:
 - Worst-Case: $O(n)$ operations
 - Useless for Inequality Queries

Key Issue: Distributed Indexing

Hash Function



Key Issue: Distributed Indexing

Hash Function

SHA-1(x)



(Secure Hash Algorithm)

Distributed Hash Table

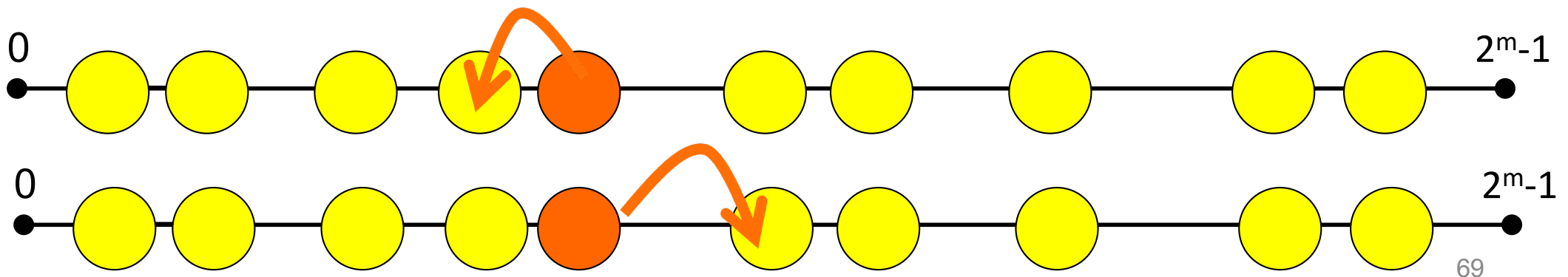
Keys & Nodes to IDs

- Keys and nodes are represented by identifiers taken from an ID space of 2^m values (m -bits space) e.g., $[0..2^{160}-1]$, all of them computed through the same hash function, e.g., SHA-1.
- For instance:
 - $ID(\text{"Let it be"}) = SHA1(\text{"Let it be"})$
 - $ID(n) = SHA1(\text{"IP address of } n\text{"})$

Distributed Hash Table

Keys & Nodes to IDs

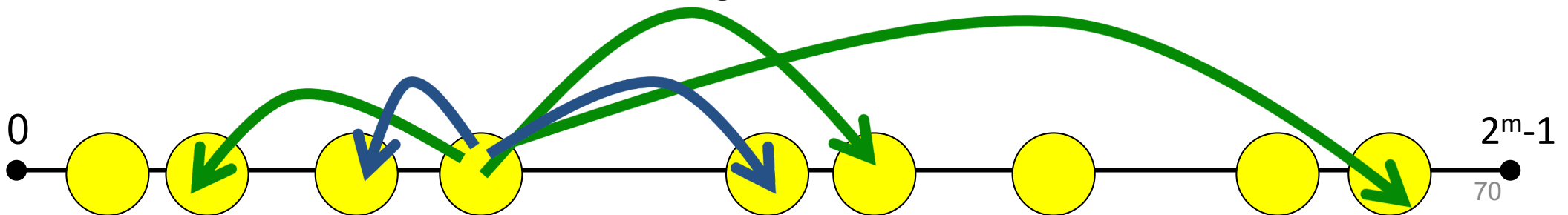
- Each node that participates to the DHT stores some keys
- A key k is stored at the node n such that, for examples:
 - $ID(n)$ is the 'closest' to $ID(k)$ (w.r.t., a given distance);
 - $ID(n)$ is the smallest node id (strictly) greater than $ID(k)$



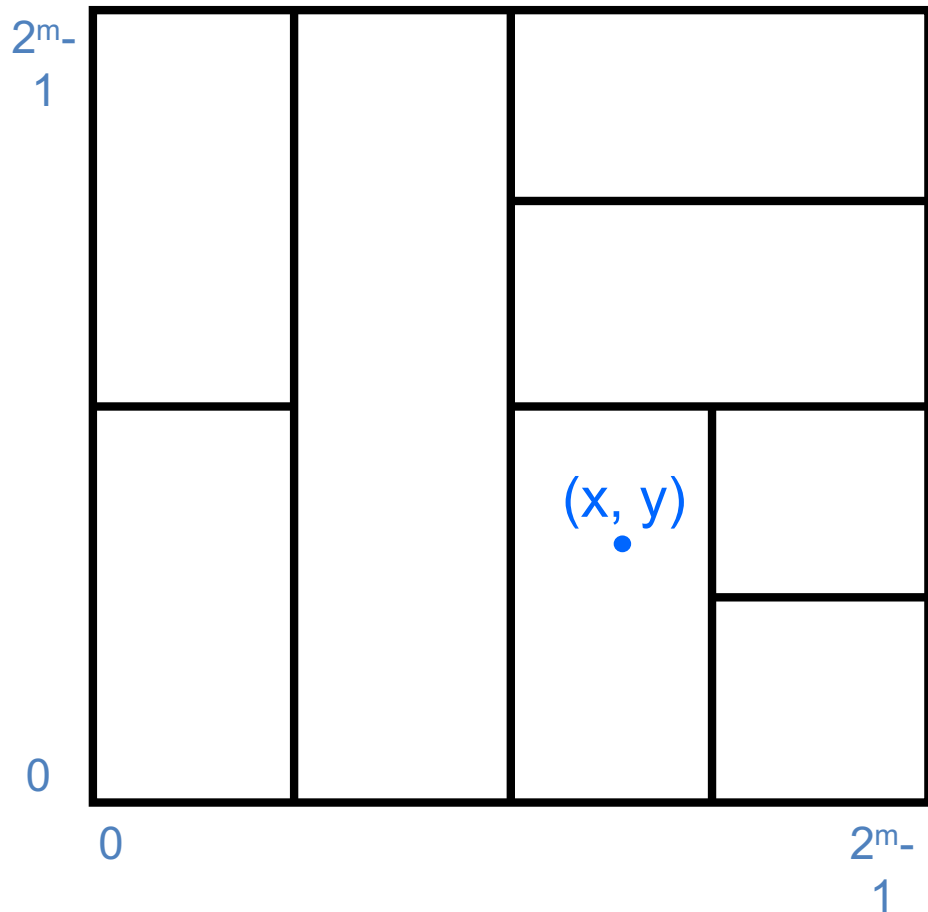
Distributed Hash Table

Build Overlay Network

- Each node has two sets of neighbors in the key space:
 - Immediate neighbors:
 - *Must avoid partition of the DHT*
 - *Must guarantee linear searching/routing*
 - *Dynamic/Fault-tolerance, self-adaptation*
 - Long-range neighbors:
 - *Allow sub-linear routing*

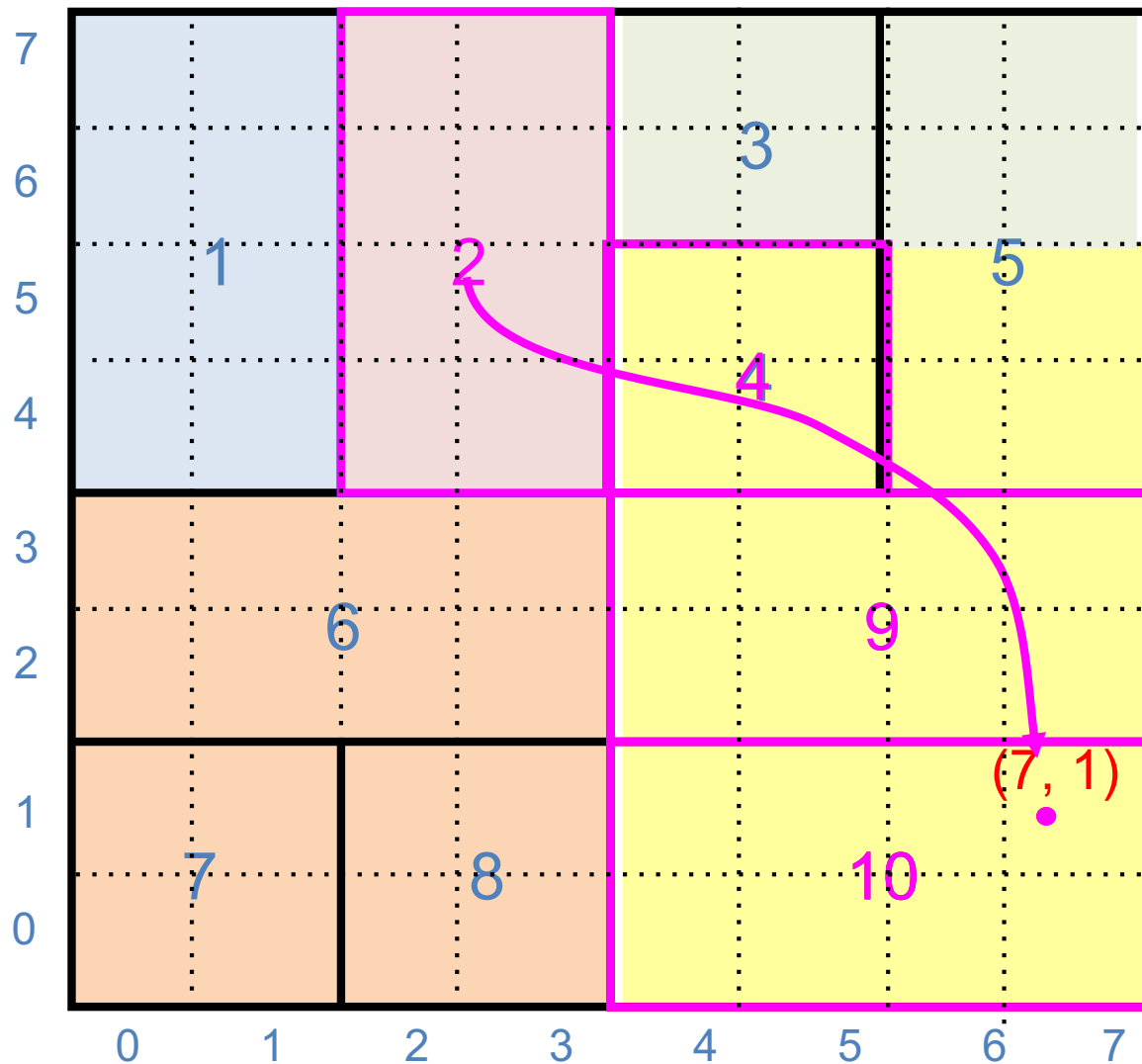


Content-Addressable Network (CAN)



- Maps an D-dimensional coordinate space
- Each node is responsible for a fraction of the space
- A **hash pair** is provided using two hash functions on the value to be stored
- Routing is done by forwarding to the neighbor that is closest in Cartesian distance

CAN example



Routing table for node 2

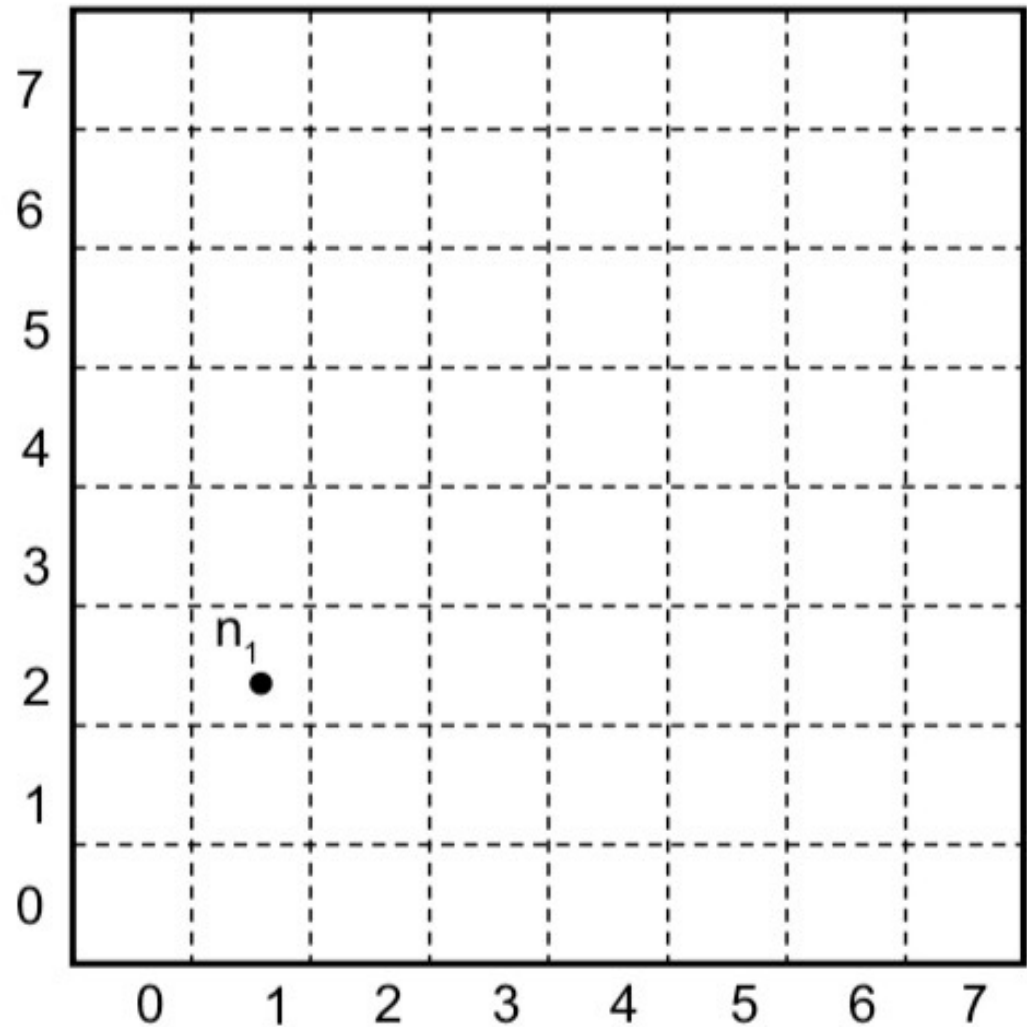
Node	Range
2	(2, 4) to (3, 7)
1	(0, 4) to (1, 7)
3	(4, 6) to (7, 7)
4	(4, 0) to (7, 5)
6	(0, 0) to (3, 3)

Entry hashed to (7, 1) is stored at node 10

Search path from 2 to (7, 1):
 $2 \rightarrow 4 \rightarrow 9 \rightarrow 10$

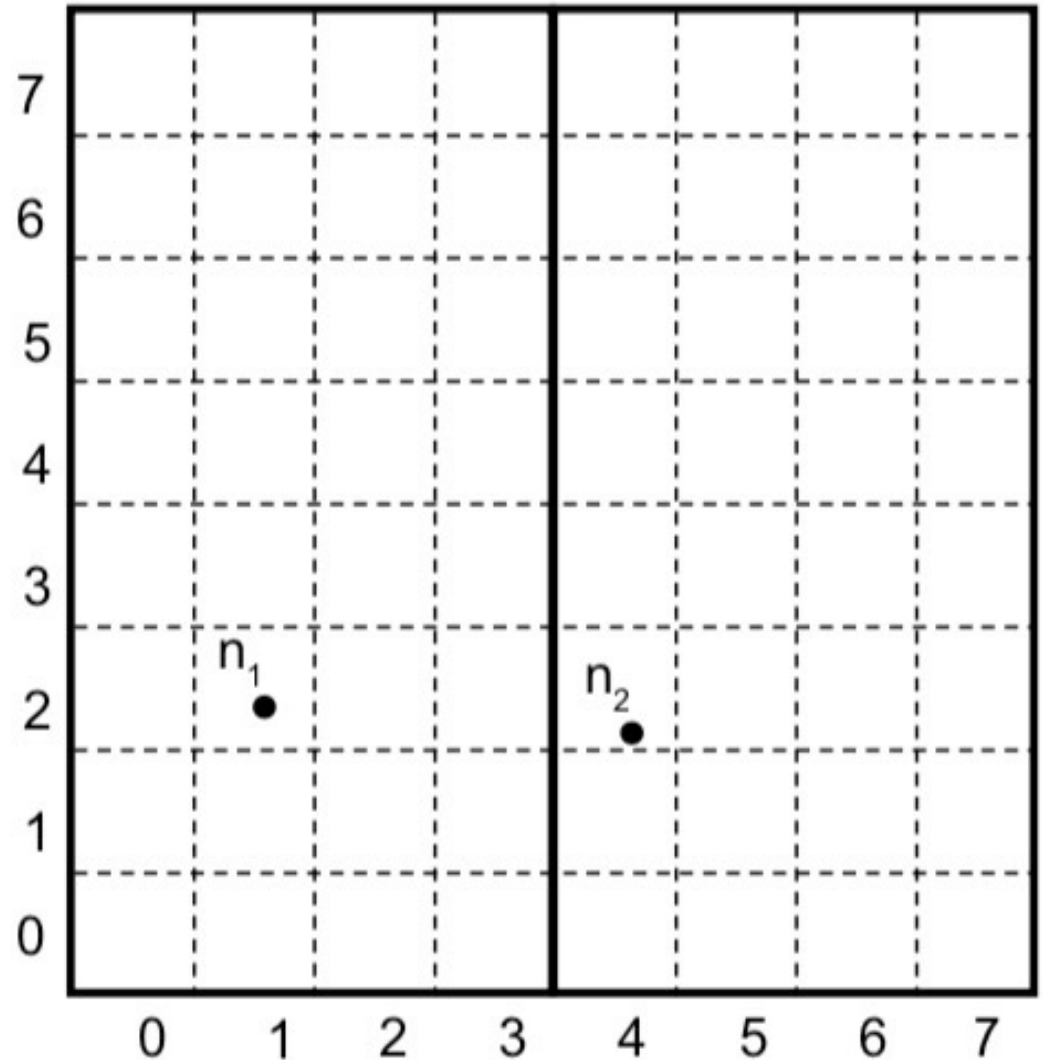
Building CAN

- The first node n_1 covers the entire space.



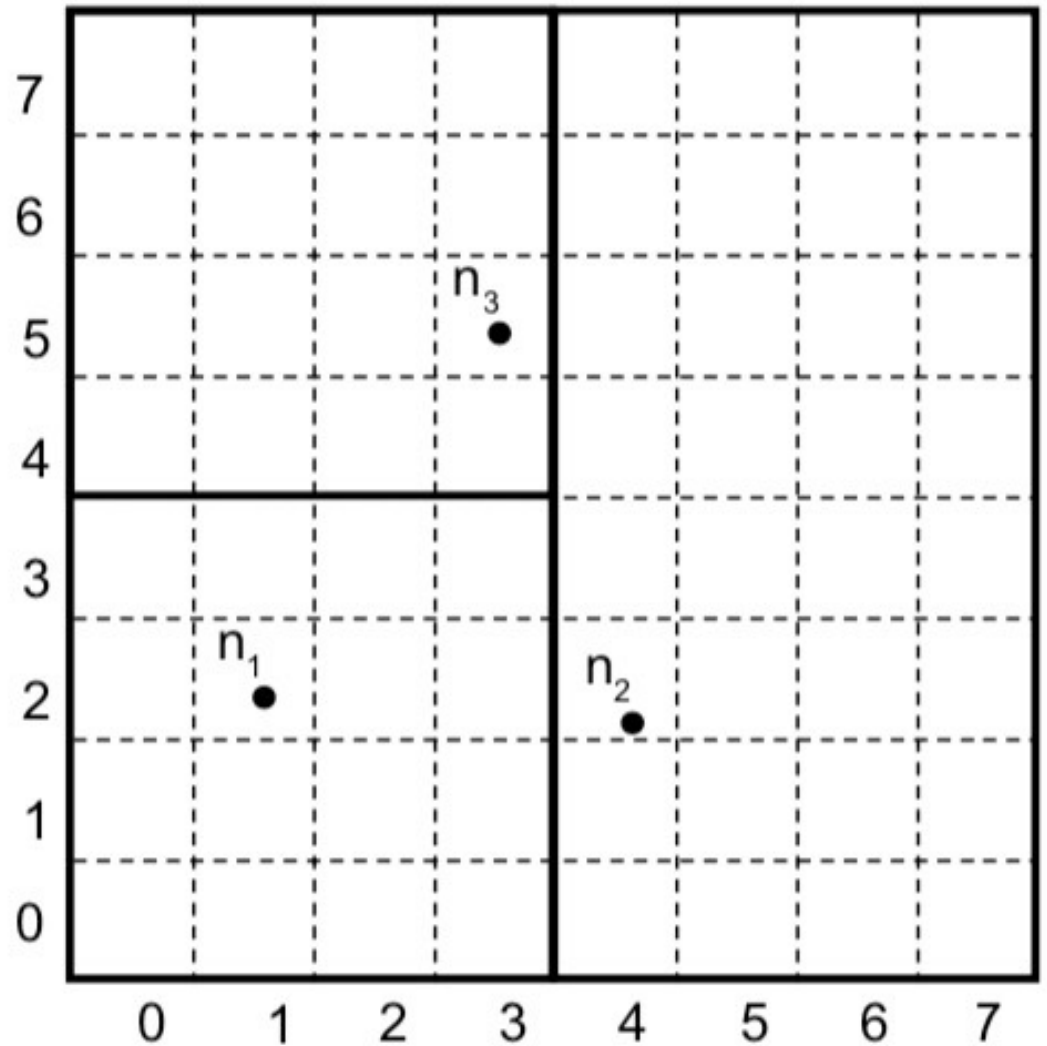
Building CAN

- Node n_2 joins and the space is divided into 2, according to n_2 ' coordinates.



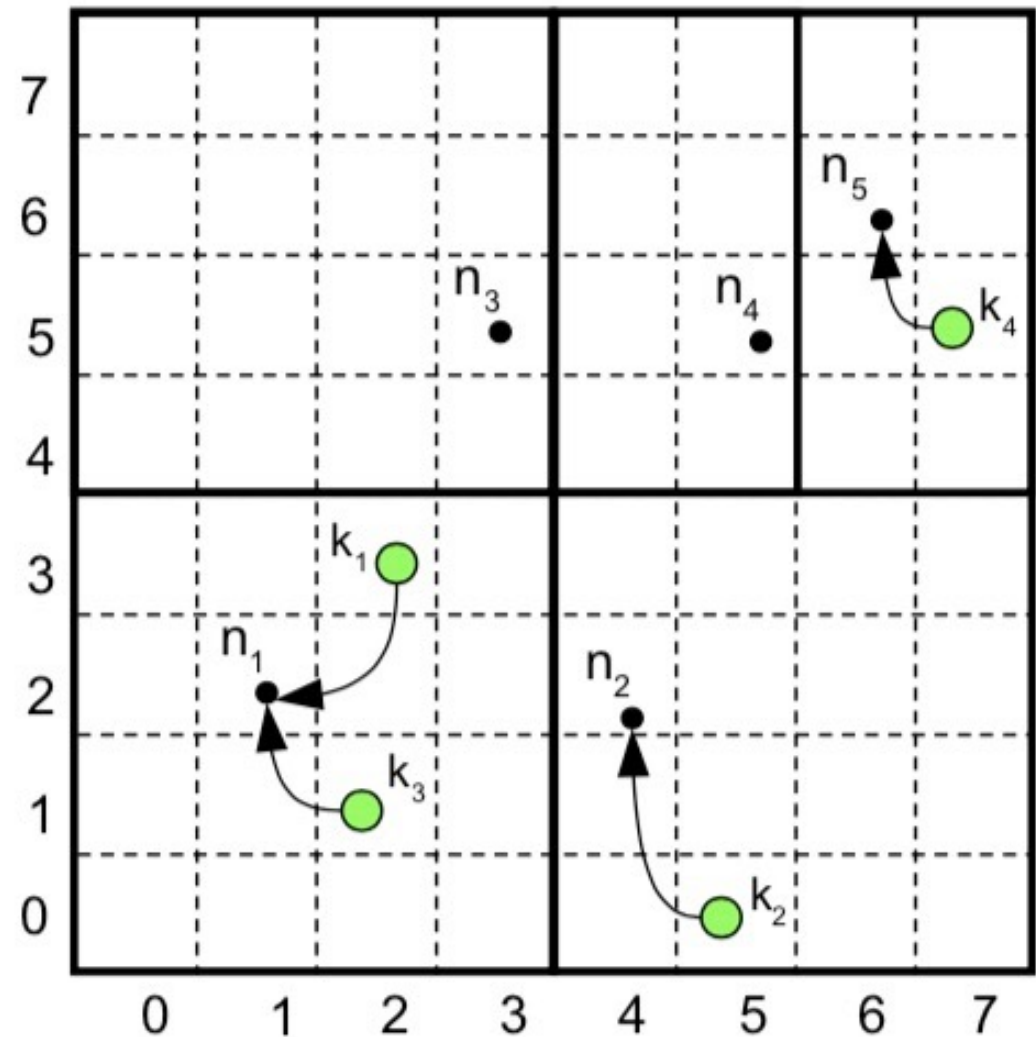
Building CAN

- Node n_3 joins with coordinates (3,5).



Building CAN

- Items are stored on nodes according to their coordinates.

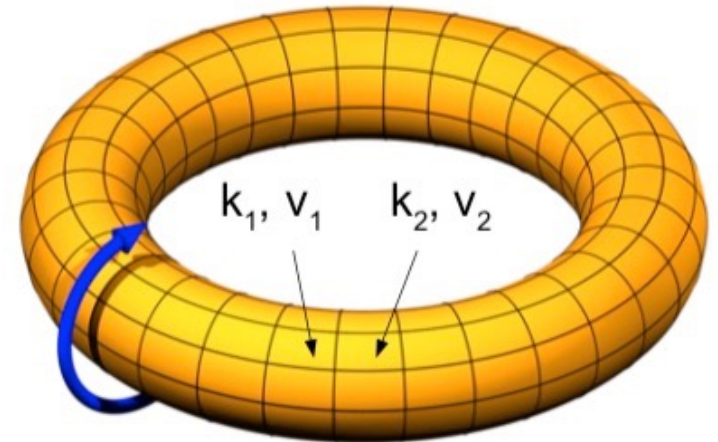


CAN joins and departures

- Joining node picks a random pair (X, Y) and contacts node A
 - A sends a join message to B, the node responsible for the pair
 - B divides its space with A and shares its routing information appropriately
 - A and B contact all its neighbors with updated info
- Departing node contacts its neighbors and finds one that can manage its space
 - The node sends updated routing info to its neighbors

CAN features

- Actually, a torus...
- Simple to understand and thus
- simple to add features to:
 - D-dimensional space (not just 2)
 - Dynamic division of space for load balancing
- Hop #: $O(Dn^{1/D})$
- Routing Table: $O(D)$
- Inequality Searches



Tapestry/Pastry

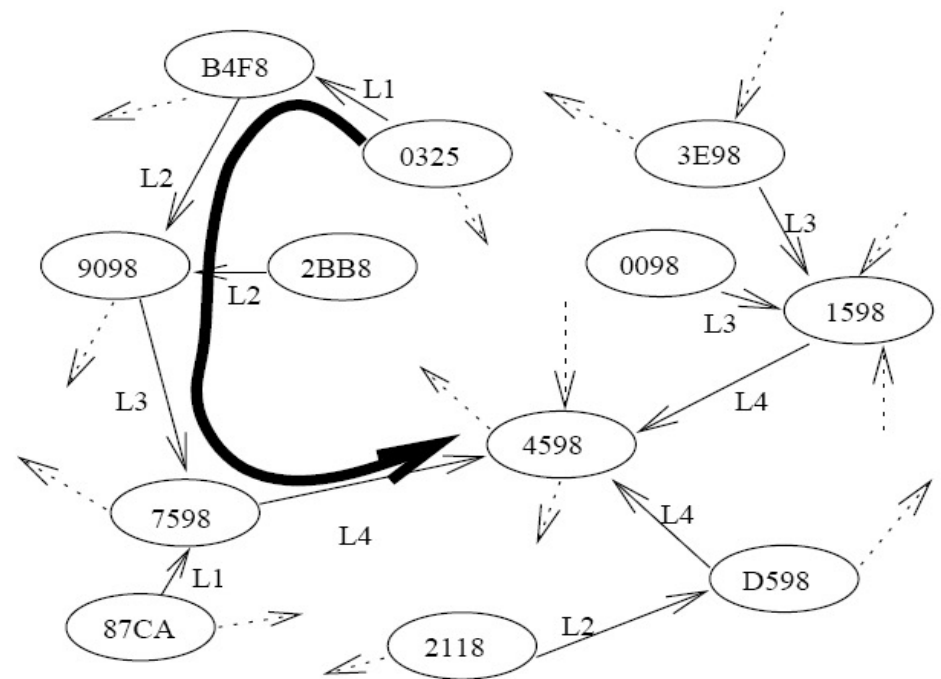
- Both based on **Plaxton Mesh**
 - Nodes – Act as routers, clients and servers simultaneously.
 - Objects – Stored on servers, searched by clients.
- Objects and nodes have **unique, location independent names**
 - Random fixed-length bit sequence in some base
 - Typically, hash of hostname or of object name
 - A hashing function h assigns each node and key a m -bit identifier

Key="LetItBe" $\xrightarrow{h}$ ID=60
IP="157.25.10.1" $\xrightarrow{h}$ ID=101

- What Plaxton does:
 - Given **message M** and **destination D**, a Plaxton mesh **routes M** to the node whose name is **numerically closest to D**

Plaxton Mesh Routing

- Routing done incrementally, digit by digit
 - In each step, message sent to node with address one digit closer to destination.
- Example - node 0325 sends M to node 4598
 - Possible route:
 - 0325
 - B4F8
 - 9098
 - 7598
 - 4598

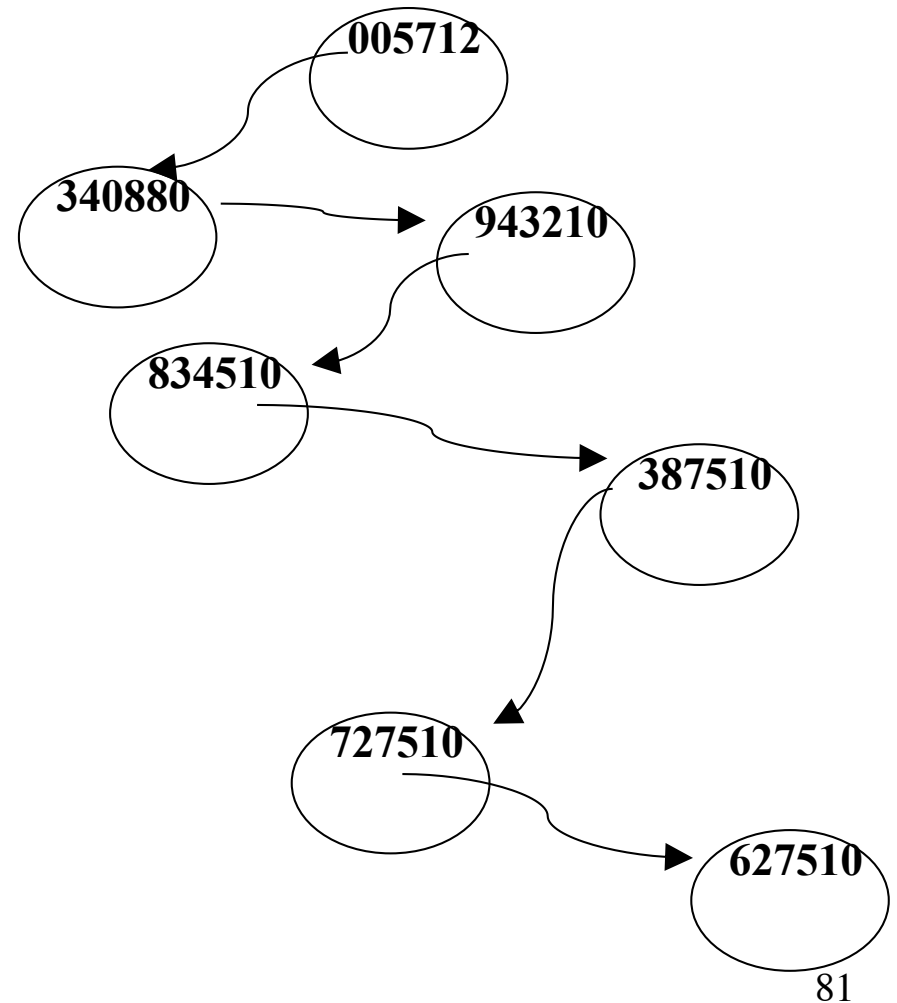


Plaxton Mesh

Routing : Another Example

Consider 2^{18} namespace, **005712** $\rightarrow$ **627510**

005712		0		1		2		3		4		5		6		7
		↓														
340880		0		1		2		3		4		5		6		7
				↓												
943210		0		1		2		3		4		5		6		7
										↓						
834510		0		1		2		3		4		5		6		7
															↓	
387510		0		1		2		3		4		5		6		7
						↓										
627510		0		1		2		3		4		5		6		7



Plaxton Mesh Routing Tables

- Each node has a neighbor map
 - Used to find a node whose address has one more digit in common with the destination
- Size of the map: $b \cdot \log_b N$, where b is the base used for the address

Plaxton Mesh Routing Tables

- x are wildcards. Can be filled with any address.
 - Each slot can have several addresses. Redundancy.
- Example: node 3642 receives message for node 2342
 - Common prefix: **XX42**
 - Look into second column.
 - Must send M to a node one digit “closer” to the destination.
 - Any host with an address like X342.

Node 3642

0642	x042	xx02	xxx0
1642	x142	xx12	xxx1
2642	x242	xx22	xxx2
3642	x342	xx32	xxx3
4642	x442	xx42	xxx4
5642	x542	xx52	xxx5
6642	x642	xx62	xxx6
7642	x742	xx72	xxx7

Plaxton Mesh Routing Algorithm

Node 3642

Assume that the destination node is a tree root. To route to node (xyz)

- Let **shared suffix = n** so far
- Look at **level n+1**
- Match the **next digit** of the destination id
- Send the message to that node

0642	x042	xx02	xxx0
1642	x142	xx12	xxx1
2642	x242	xx22	xxx2
3642	x342	xx32	xxx3
4642	x442	xx42	xxx4
5642	x542	xx52	xxx5
6642	x642	xx62	xxx6
7642	x742	xx72	xxx7

Pastry: Routing procedure

```
If (destination is within range of our leaf set)
    forward to numerically closest member
else
    let  $l$  = length of shared prefix
    let  $d$  = value of  $l$ -th digit in  $D$ 's address
    if ( $R_l^d$  exists)
        forward to  $R_l^d$ 
    else
        forward to a known node that
        (a) shares at least as long a prefix
        (b) is numerically closer than this node
```

Plaxton Mesh

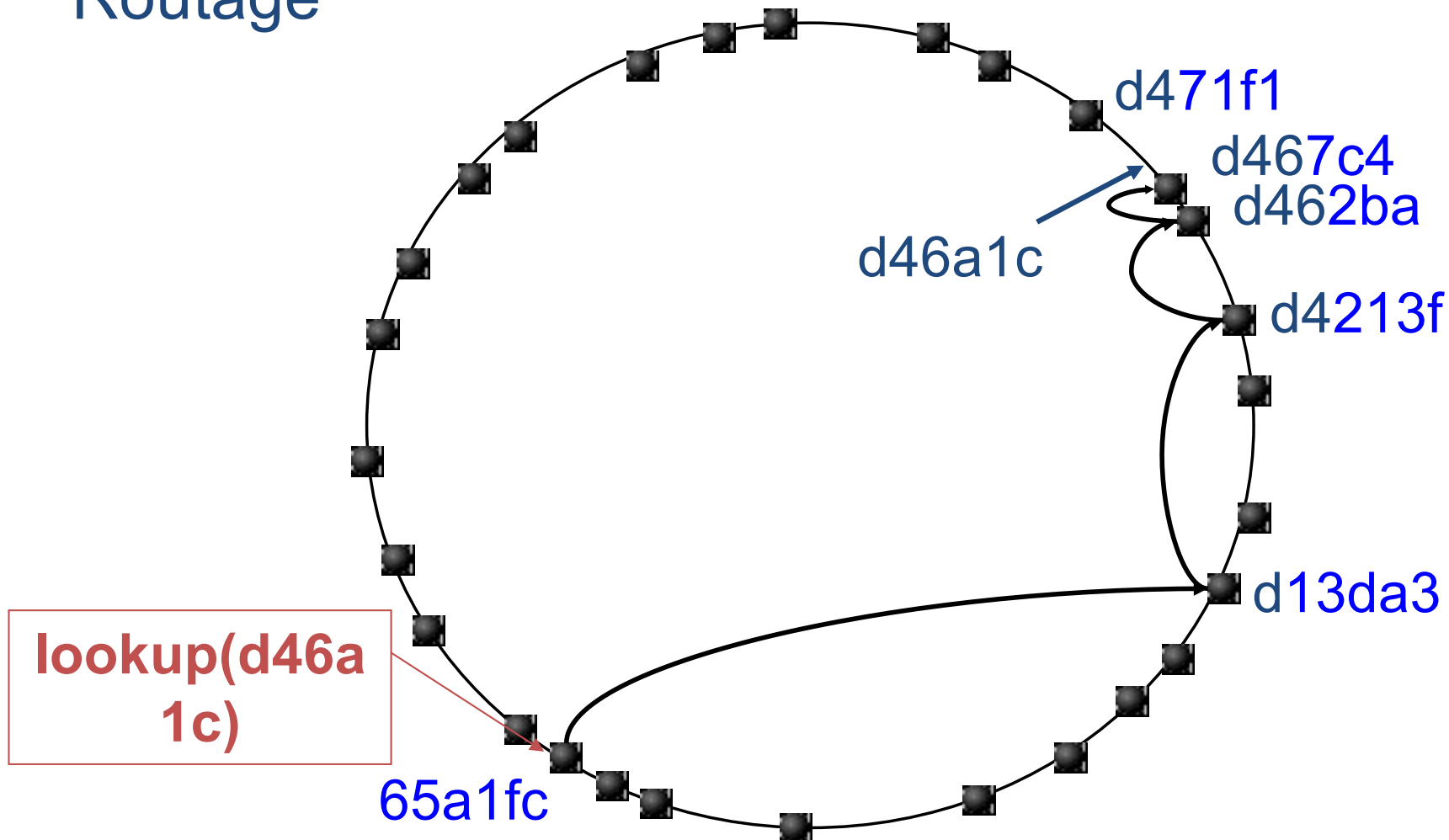
Main Characteristics

- Memory per node : $O(\log(N))$
- Self-adaptative
- Rooting in $O(\log(N))$ hops
- Fault-tolerant
- Pastry, Patestry : Very similar to *Chord*

Plaxton Mesh

Main Characteristics

Routage



Chord Protocol

- A consistent hashing function (SHA-1) assigns each node and key a m -bit identifier

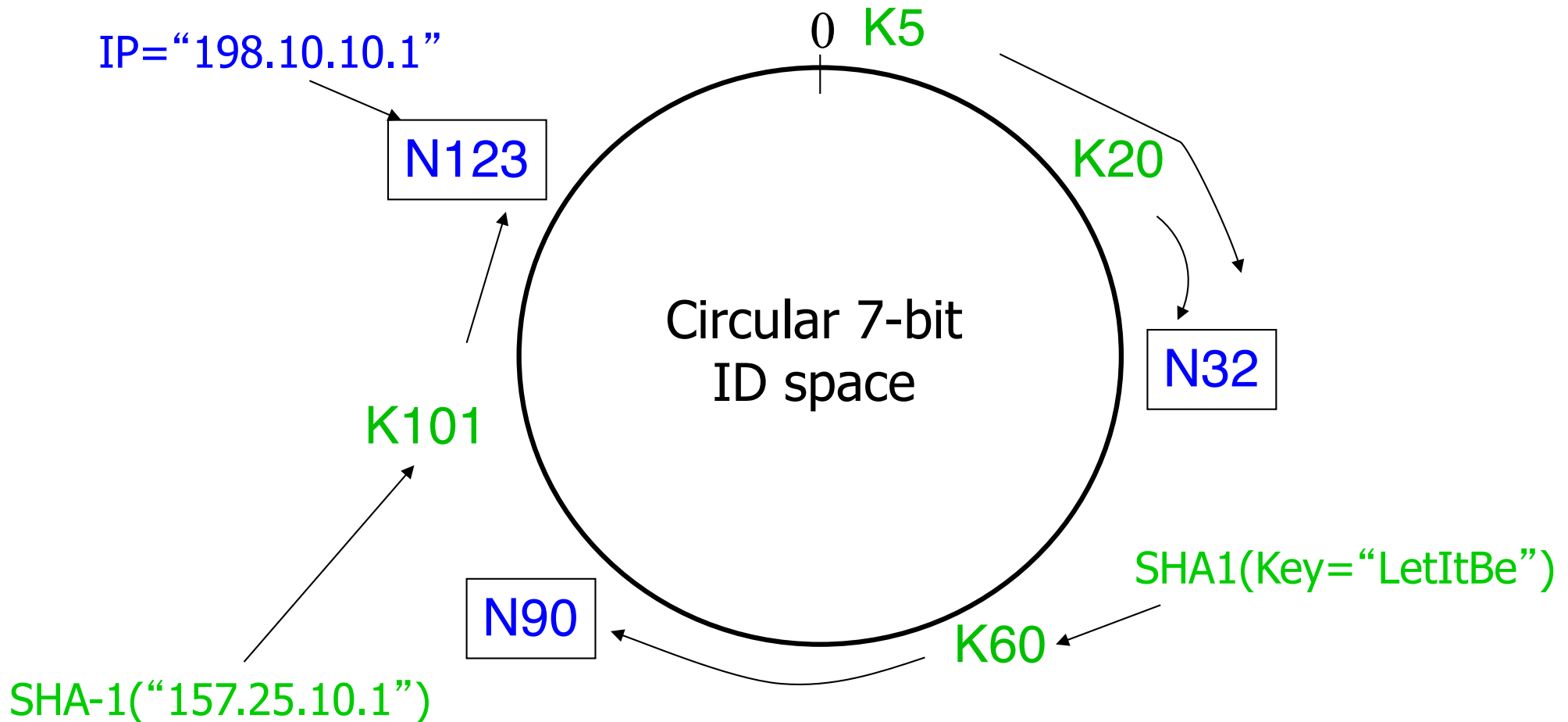
Key="LetItBe" $\xrightarrow{\text{SHA-1}}$ ID=60

IP="157.25.10.1" $\xrightarrow{\text{SHA-1}}$ ID=101

- Identifiers are ordered on a identifier circle modulo 2^m called a chord ring.
- $\text{successor}(k)$ = first node whose identifier is $\geq$ identifier of k in identifier space.

Consistent Hashing (cont.)

- A key is stored at its successor: node with next higher ID

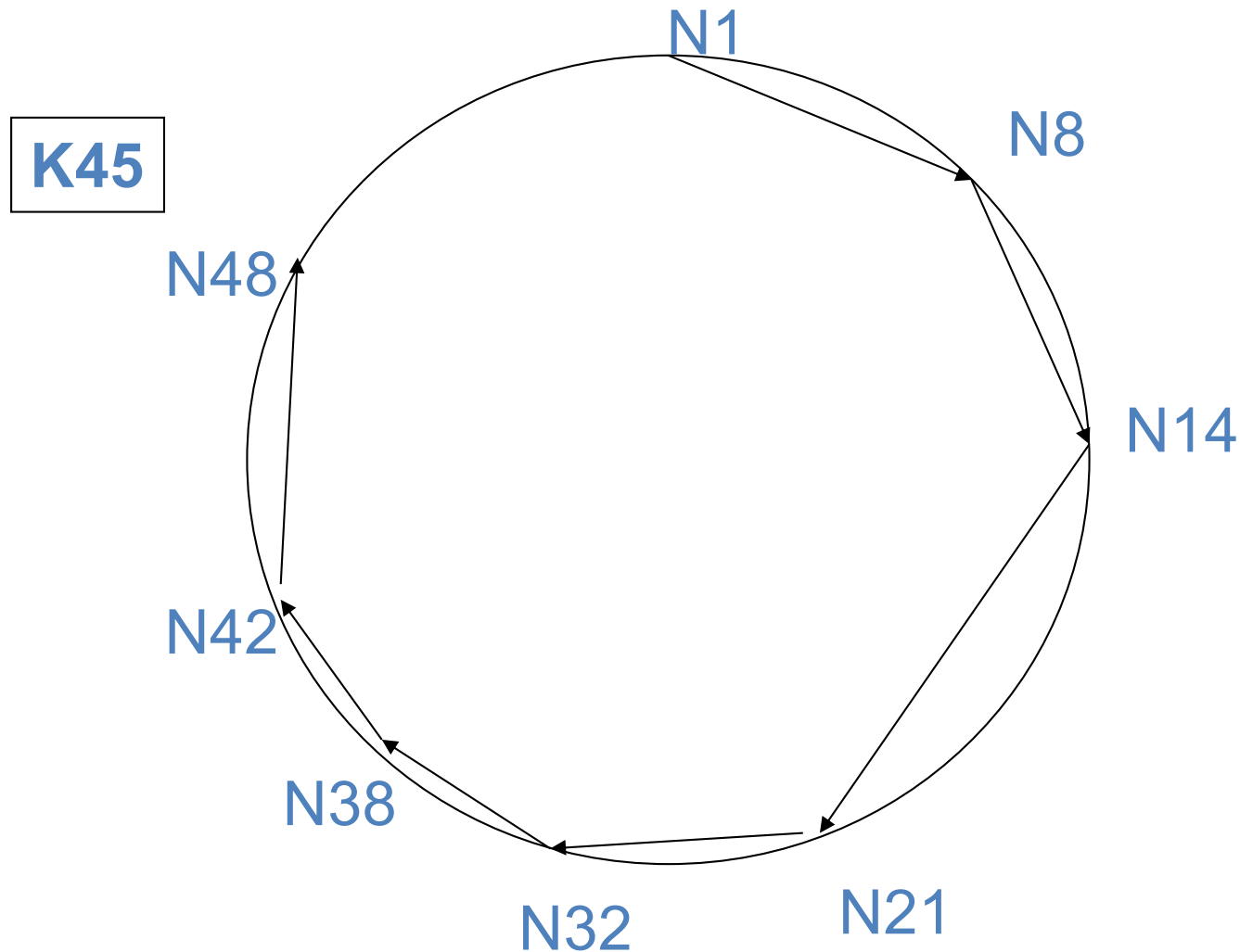


Theorem

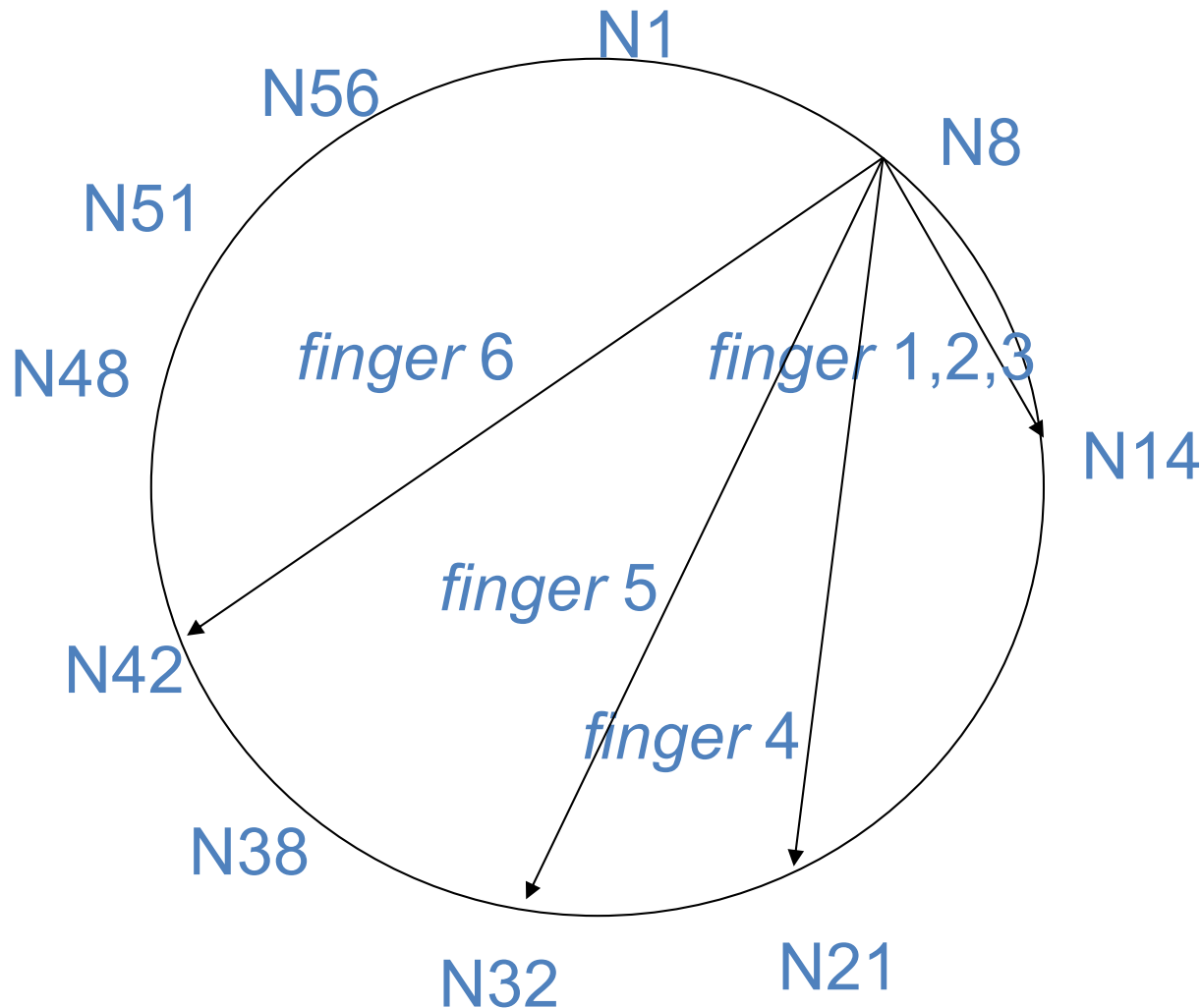
- For any set of N nodes and K keys, with *high probability*:
 1. Each node is responsible for at most $(1+e)K/N$ keys.
 2. When an $(N+1)^{\text{st}}$ node joins or leaves the network, responsibility for $O(K/N)$ keys changes hands.

$$e = O(\log N)$$

Simple Key Location Scheme



Scalable Lookup Scheme

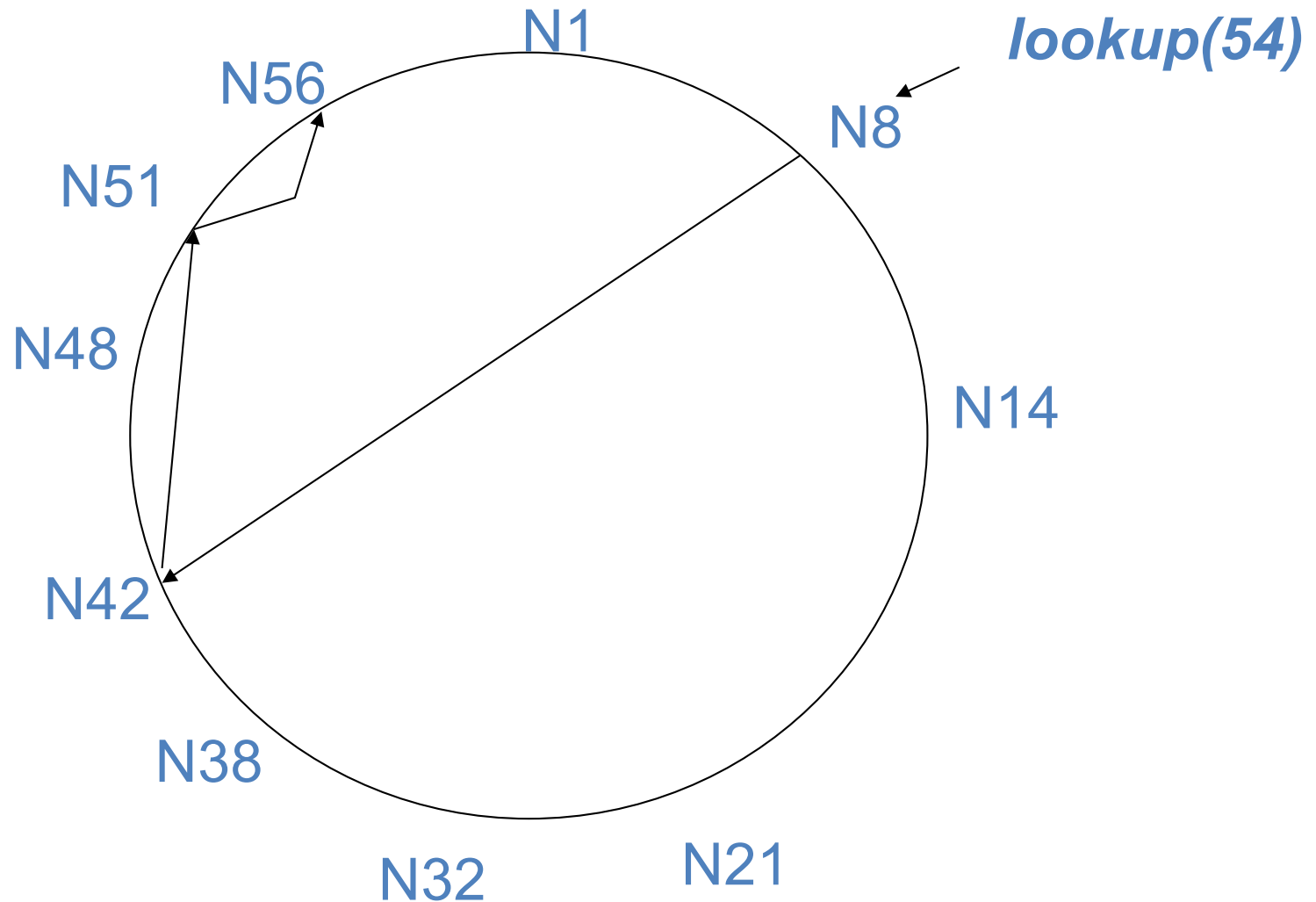


Finger Table for N8

N8+1	N14
N8+2	N14
N8+4	N14
N8+8	N21
N8+16	N32
N8+32	N42

finger [k] = first node that succeeds $(n+2^{k-1}) \bmod 2^m$

Lookup Using Finger Table



Scalable Lookup Scheme

// ask node n to find the successor of id

n.find_successor(id)

if (id belongs to (n, successor])

return successor;

else

n0 = closest preceding node(id);

return n0.find_successor(id);

// search the local table for the highest predecessor of id

n.closest_preceding_node(id)

for i = m **downto** 1

if (finger[i] belongs to (n, id))

return finger[i];

return n;

Scalable Lookup Scheme

- Each node forwards query at least halfway along distance remaining to the target
- Theorem: With high probability, the number of nodes that must be contacted to find a successor in a N -node network is $O(\log N)$

Chord joins and departures

- Joining node gets an ID N and contacts node A
 - A searches for N 's predecessor B
 - N becomes B 's successor
 - N contacts B and gets its successor pointer and finger table, and updates the finger table values
 - N contacts its successor and inherits keys
- Departing node contacts its predecessor, notifies it of its departure, and sends it the new successor pointer

Chord features

- Correct in the face of routing failures since can correctly route with successor pointers
- Replication can easily reduce the possibility of losing data
- Caching can easily place data along a likely search path for future queries
- Simple to understand

Classical & Future Issues

Classical Issues

- Large Scaling Search (Unstructured)
 - Gossip
- Data Replication and Maintenance
- Data Integrity
- Data Persistence
- Security / Anonymity
- Attacks
 - Sybil / Byzantin attack

Open Problems

- CLOUD Computing
 - Shared/Cooperative Work
- Deployment
- Resource Allocation
- Observation / Performance evaluation
- Security concerns / Building trust systems